



UAI
UNIVERSIDAD ADOLFO IBÁÑEZ
FACULTAD DE INGENIERÍA Y CIENCIAS

INTRODUCCIÓN A PYTHON

INTRODUCCIÓN

Background designed by kjpargeter / Freepik

Miguel Carrasco
miguel.carrasco@uai.cl
2025



2004

- ▶ Ingeniero Civil en Informática, Universidad de Santiago de Chile,
- ▶ Magíster en Informática Universidad de Santiago de Chile, *distinción máxima*



2008

- ▶ Magíster en Ingeniería Pontificia Universidad Católica de Chile.



2010

- ▶ Docteur en Informatique, Université Pierre et Marie Curie (Paris VI, Sorbonne University, Francia) *distinción máxima*
- ▶ Doctor en Ciencias de la Ingeniería, mención computación, Pontificia Universidad Católica de Chile.



2013

- ▶ Post-doc, Institut de la vision, Paris, France.



2023

- ▶ International Engineering Educator ING.PAED.IGIP
International Society for Engineering Pedagogy

Áreas de trabajo

- Académico, Escuela de Informática y Telecomunicaciones, Facultad de Ingeniería y Ciencias, Universidad Diego Portales.
- Sus áreas de interés son la inspección automática, visión por computador, procesamiento de imágenes, interfaz humano computador, Internet of Things.



Guido Van Rossum

(creador de Python)

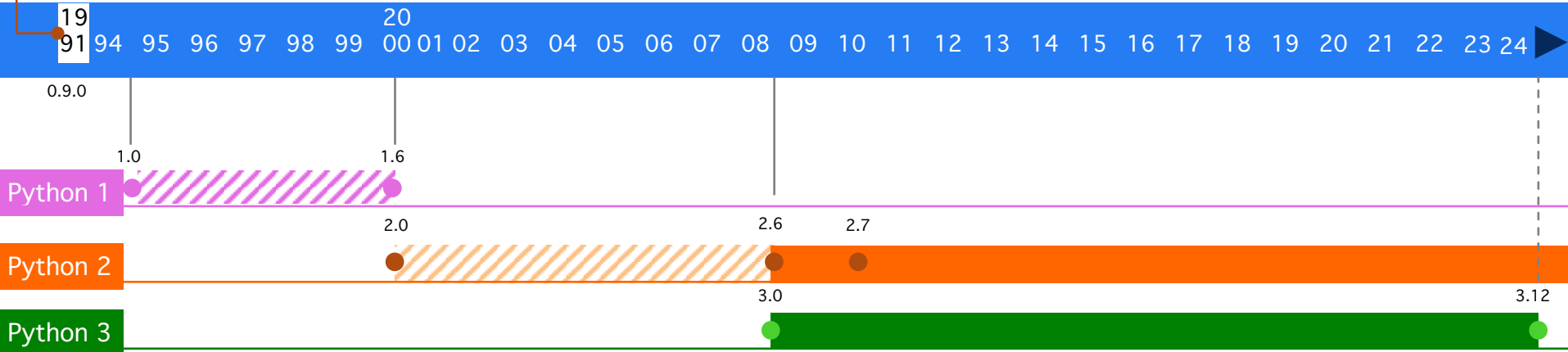
“es un científico de la computación conocido por ser el autor del lenguaje de programación Python en 1991”
(fuente: Wikipedia)



Python es un lenguaje de propósito general y de tipo script. Es empleado para aplicaciones de interfaces, bases de datos, computación científica, aplicaciones web (client-server web programming) entre otras.

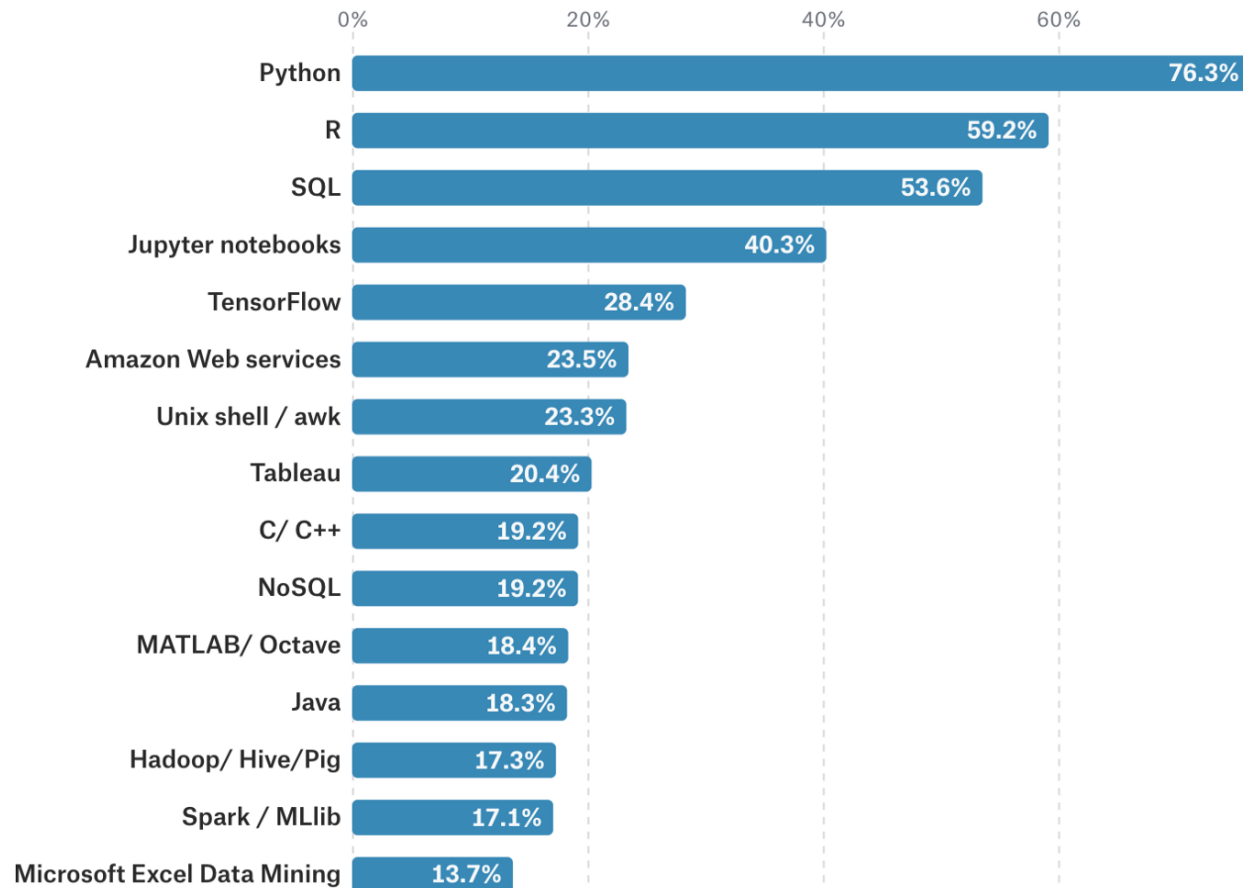
Principales características:

- Facilidad de lectura (código interpretado)
- Sistema de tipo dinámico
- Manejo automático de administración de memoria



2022

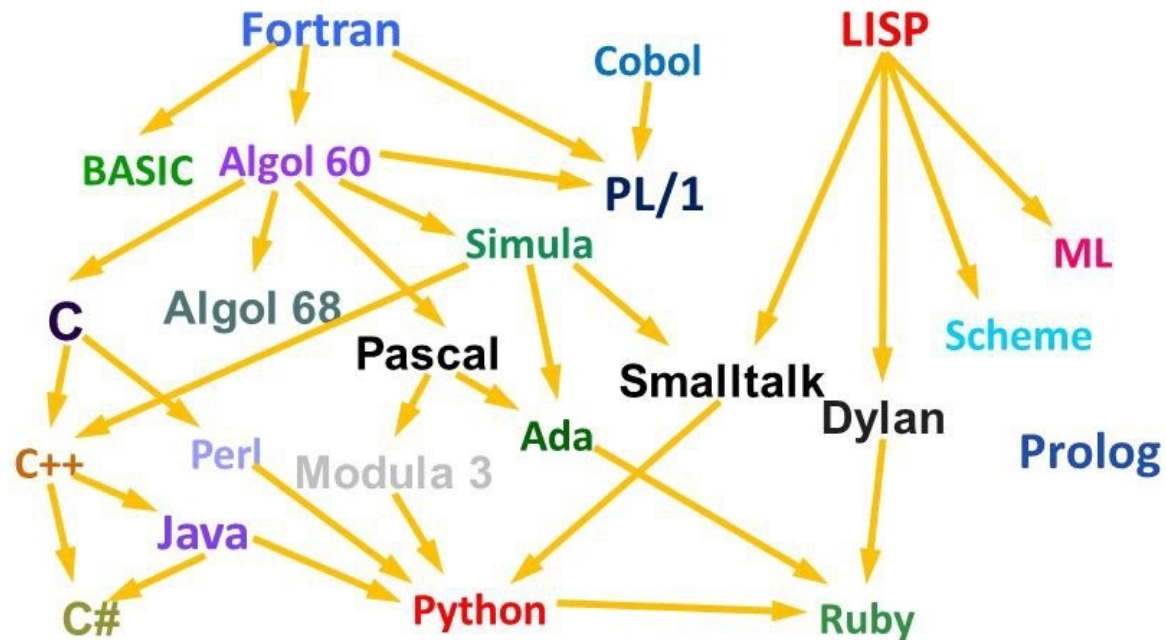
What tools are used at work?"



fuelle: <https://towardsdatascience.com/training-your-staff-in-data-science-heres-how-to-pick-the-right-programming-language-dda349354b18>

A family tree of languages

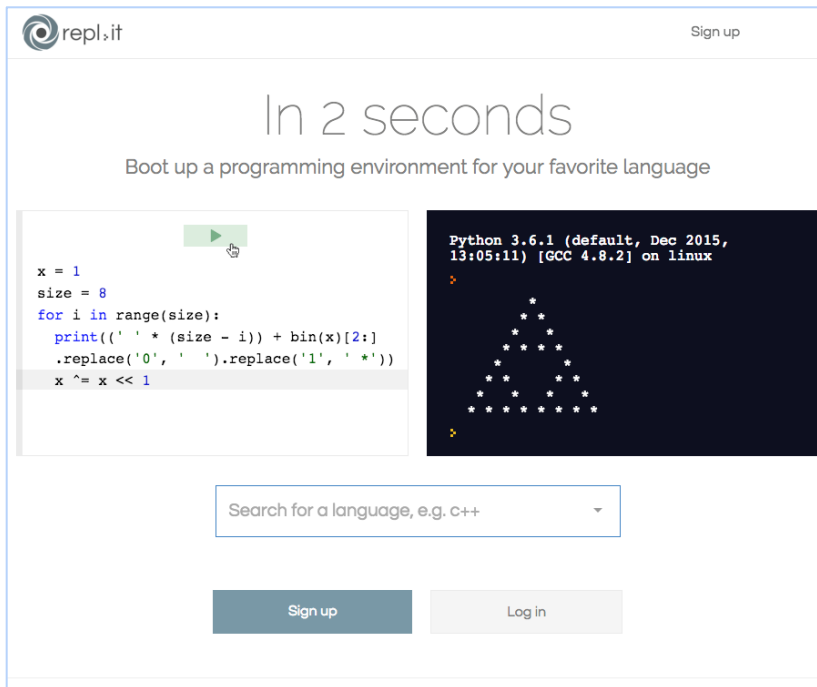
Some of the 2400 + programming languages



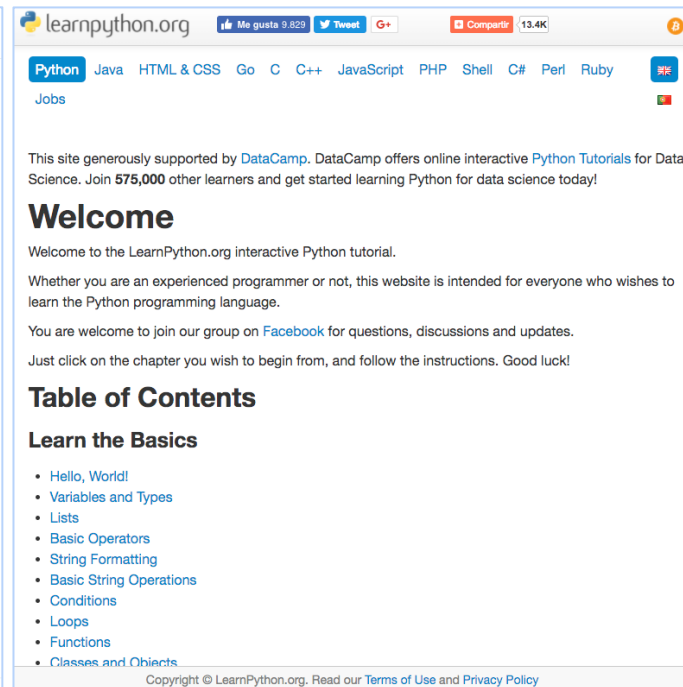
fuentes: History of Programming Languages by Ursula Lewis

online

► Ambientes de trabajo



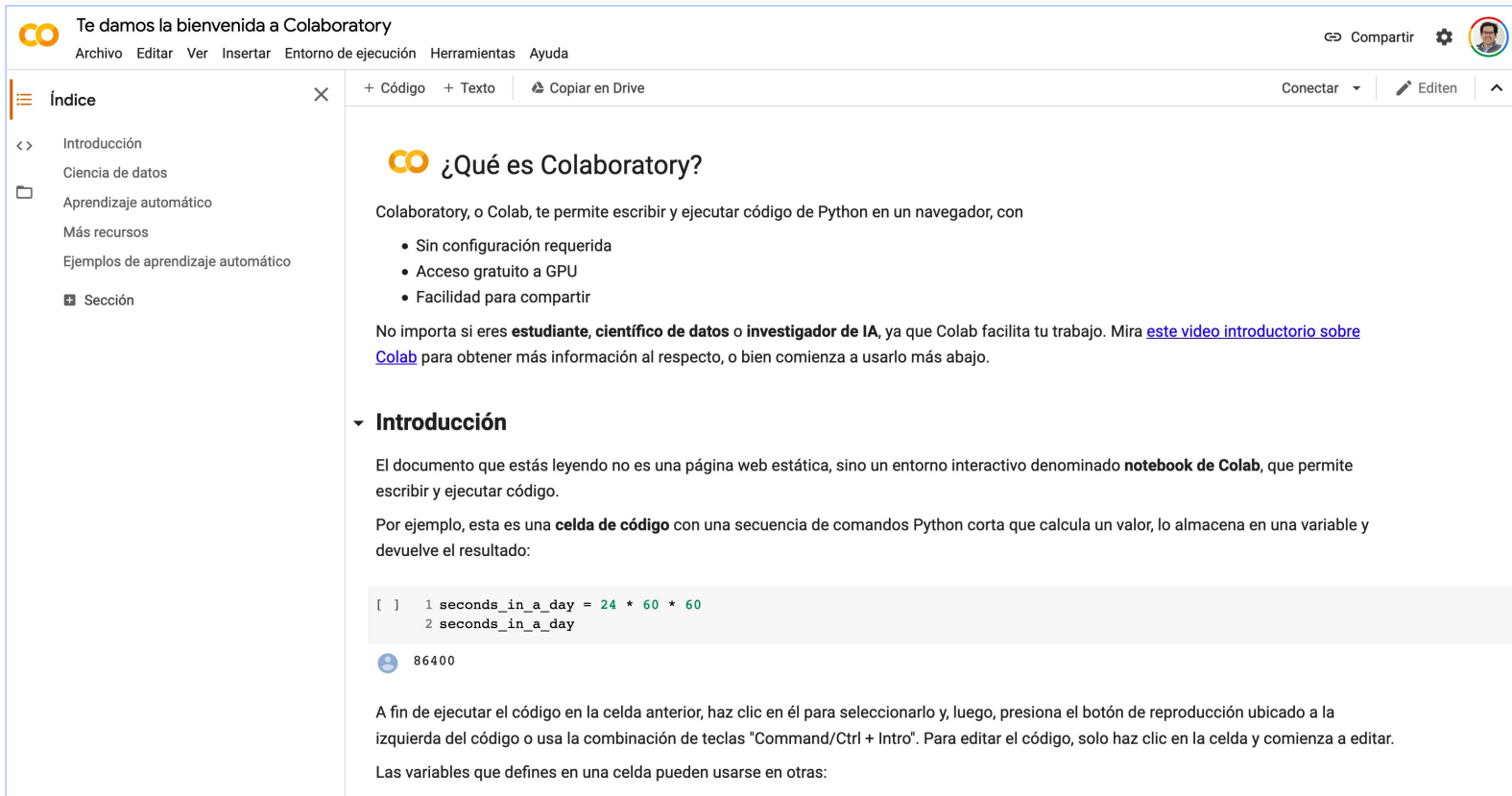
► repl.it



► www.learnpython.org

online

► Ambientes de trabajo



Te damos la bienvenida a Colaboratory

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda

Compartir Configuración Perfil

Índice

- Introducción
- Ciencia de datos
- Aprendizaje automático
- Más recursos
- Ejemplos de aprendizaje automático
- Sección

+ Código + Texto Copiar en Drive

Conectar Editen

¿Qué es Colaboratory?

Colaboratory, o Colab, te permite escribir y ejecutar código de Python en un navegador, con

- Sin configuración requerida
- Acceso gratuito a GPU
- Facilidad para compartir

No importa si eres **estudiante**, **científico de datos** o **investigador de IA**, ya que Colab facilita tu trabajo. Mira [este video introductorio sobre Colab](#) para obtener más información al respecto, o bien comienza a usarlo más abajo.

Introducción

El documento que estás leyendo no es una página web estática, sino un entorno interactivo denominado **notebook de Colab**, que permite escribir y ejecutar código.

Por ejemplo, esta es una **celda de código** con una secuencia de comandos Python corta que calcula un valor, lo almacena en una variable y devuelve el resultado:

```
[ ] 1 seconds_in_a_day = 24 * 60 * 60
    2 seconds_in_a_day
```

86400

A fin de ejecutar el código en la celda anterior, haz clic en él para seleccionarlo y, luego, presiona el botón de reproducción ubicado a la izquierda del código o usa la combinación de teclas "Command/Ctrl + Intro". Para editar el código, solo haz clic en la celda y comienza a editar.

Las variables que defines en una celda pueden usarse en otras:

► <https://colab.research.google.com/>

offline

Python

PSF

Docs

PyPI

Jobs

Community



GO

Socialize

About

Downloads

Documentation

Community

Success Stories

News

Events

Download the latest version for Mac OS X

Download Python 3.6.4

Download Python 2.7.14

Wondering which version to use? [Here's more about the difference between Python 2 and 3.](#)

Looking for Python with a different OS? Python for [Windows](#), [Linux/UNIX](#), [Mac OS X](#), [Other](#)

Want to help test development versions of Python? [Pre-releases](#)



offline



Products

Why Anaconda?

Solutions

Resources

Company

Download



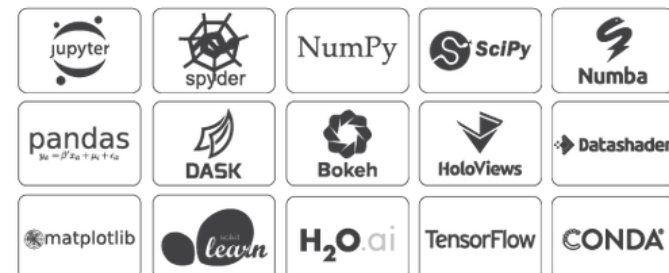
Anaconda Distribution

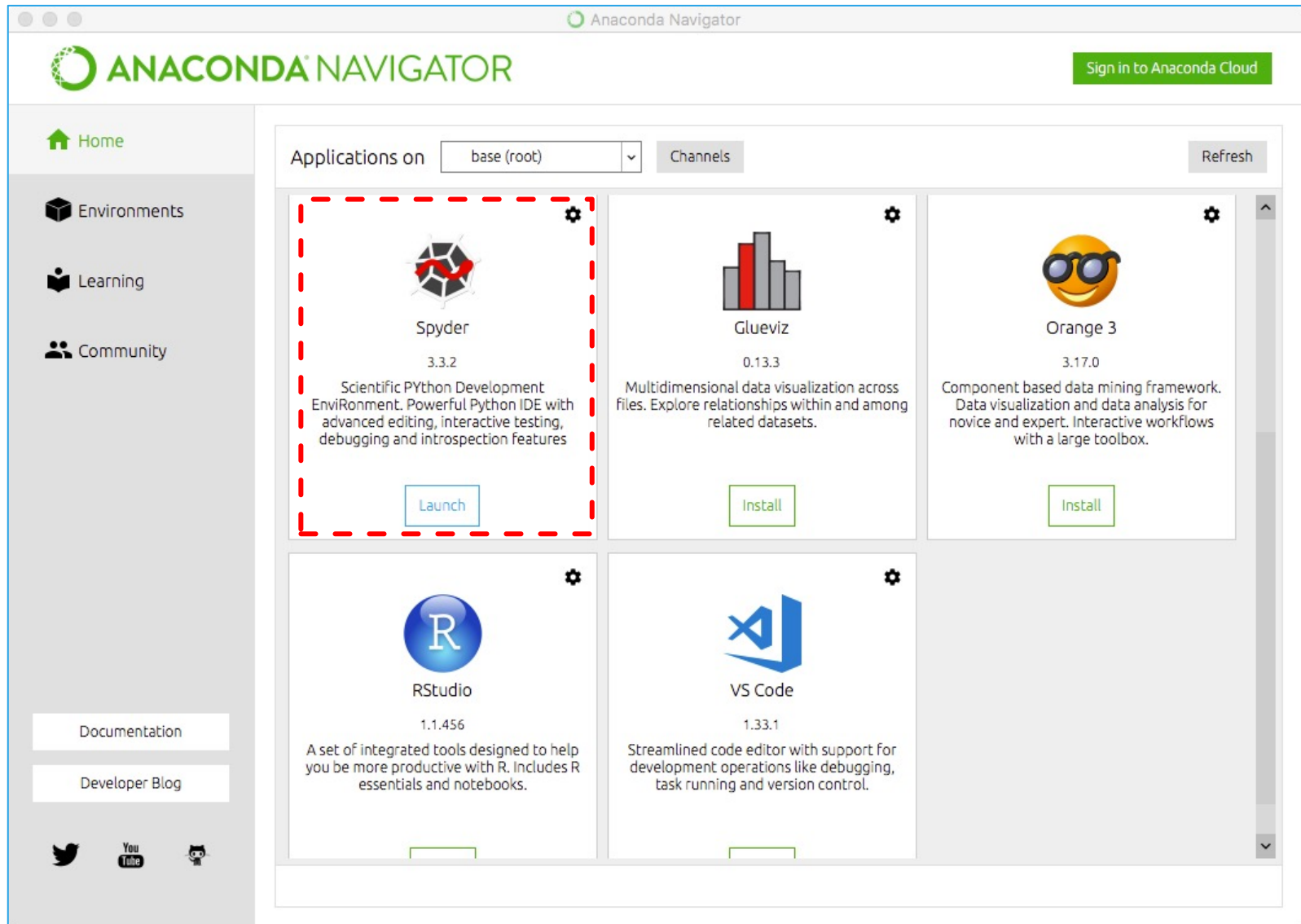
The World's Most Popular Python/R Data Science Platform

Download

The open-source **Anaconda Distribution** is the easiest way to perform Python/R data science and machine learning on Linux, Windows, and Mac OS X. With over 11 million users worldwide, it is the industry standard for developing, testing, and training on a single machine, enabling *individual data scientists* to:

- Quickly download 1,500+ Python/R data science packages
- Manage libraries, dependencies, and environments with **Conda**
- Develop and train machine learning and deep learning models with **scikit-learn**, **TensorFlow**, and **Theano**
- Analyze data with scalability and performance with **Dask**, **NumPy**, **pandas**, and **Numba**
- Visualize results with **Matplotlib**, **Bokeh**, **Datashader**, and **Holoviews**





- ▶ Introducción a Python
- ▶ Primeros pasos
 - Lección 01: Despliegue de datos

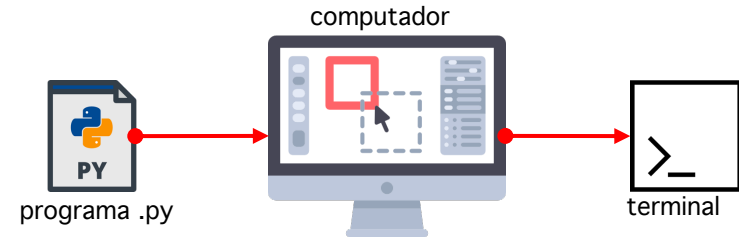


```
self.FidValue = OrderedDict(sorted(self.FidValue.items(), key=lambda item: item[0]))  
#Read item in dictionary  
for key, value in self.FidValue.items():  
    typeOfFID = mapFidType[key]  
    if (typeOfFID == "DATE"):  
        d = datetime.datetime.strptime(str(value), "%Y-%m-%d")  
        dataCal = datetime.date.strptime(str(value), "%Y-%m-%d")  
        FidAndValue = FidAndValue + value  
    else: FidAndValue = FidAndValue + value
```

```
try:  
    start = date(int(self.start_year.get()),  
                self.months.index(self.start_month.get()),  
                int(self.start_day.get()))  
  
    end = date(int(self.end_year.get()),  
              self.months.index(self.end_month.get()),  
              int(self.end_day.get()))
```



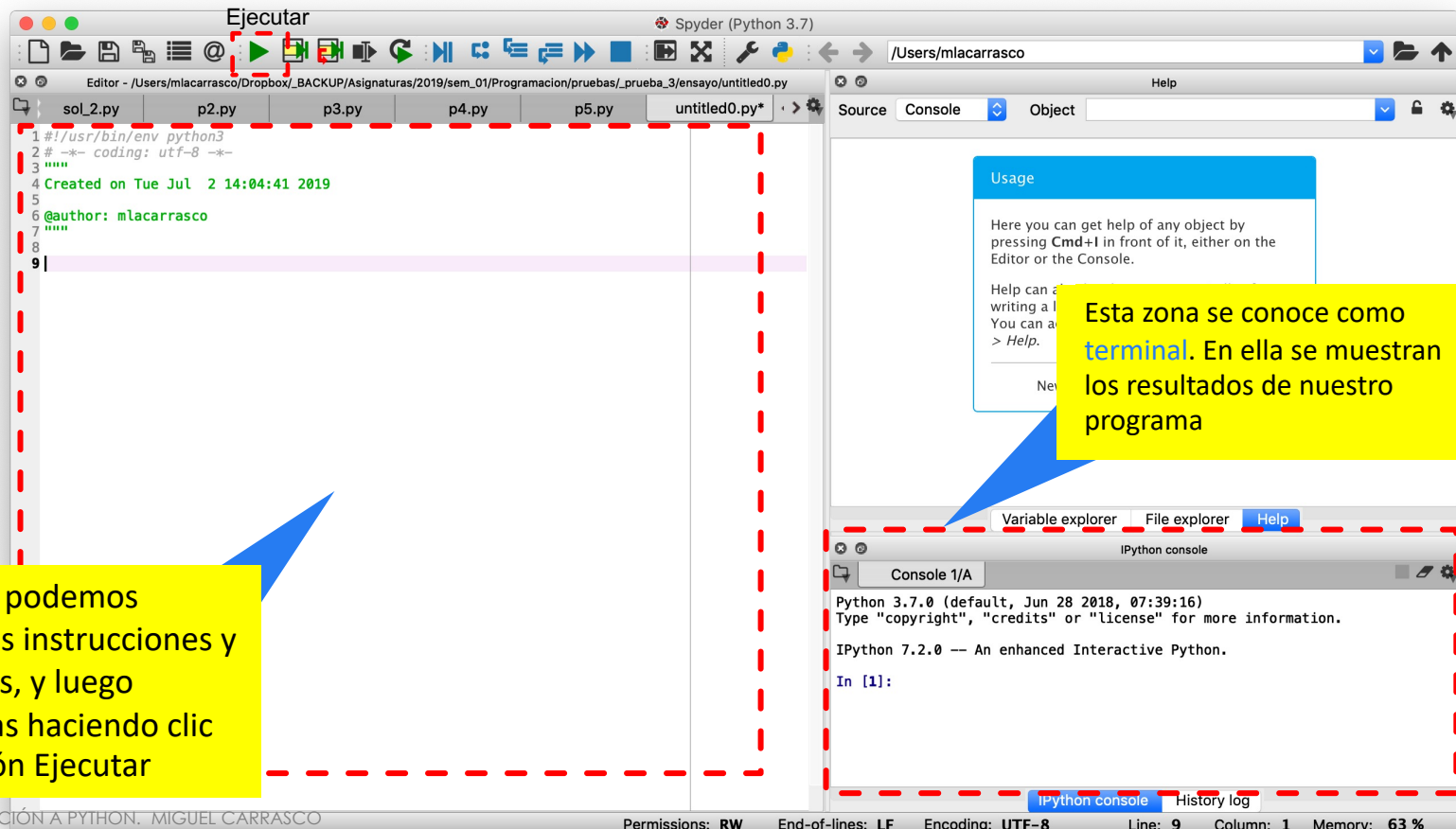
Un programa en Python está conformado por **líneas de texto** que contienen **una o más instrucciones y sentencias del lenguaje**



spyder

Esta zona podemos escribir las instrucciones y sentencias, y luego ejecutarlas haciendo clic en el botón Ejecutar

Esta zona se conoce como **terminal**. En ella se muestran los resultados de nuestro programa



▼ print()

El comando `print()` permite desplegar el resultado de un variable o un texto por la terminal. Si escribimos texto, éste debe ir escrito entre comillas. Si escribimos una variable, éste despliega el valor almacenado en dicha variable.

The screenshot shows the Spyder Python IDE interface. The code editor on the left contains the following Python code:

```
1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
3 """
4 Created on Tue Jul  2 14:04:41 2019
5
6 @author: mlacarrasco
7 """
8
9 print('hola')
```

The IPython console on the right shows the output of the code execution:

```
Python 3.7.0 (default, Jun 28 2018, 07:39:16)
Type "copyright", "credits" or "license" for more information.

IPython 7.2.0 -- An enhanced Interactive Python.

In [1]: runfile('/Users/mlacarrasco/Dropbox/_BACKUP/Asignaturas/2019/sem_01/MagisterDataScience/clases/modulo_1_control/code/test.py', wdir='/Users/mlacarrasco/Dropbox/_BACKUP/Asignaturas/2019/sem_01/MagisterDataScience/clases/modulo_1_control/code')
hola

In [2]:
```

A sequence of steps diagram is overlaid on the image, showing the process of running the code:

1. El primer paso consiste en escribir uno o más instrucciones. El segundo paso es ejecutar el código, y el último paso es revisar los resultados en la terminal.
2. Ejecutar el código (indicated by a red dashed box around the Run button in the Spyder toolbar).
3. Revisar los resultados en la terminal (indicated by a red dashed box around the IPython console output).

Seuencia de pasos:

```
graph LR
    1((1)) --> 2((2))
    2 --> 3((3))
```

▼ print()

Si escribimos texto, **éste debe ir escrito entre comillas**. En caso contrario, el computador arrojará un error ya que no sabrá donde termina la cadena de texto.

Como se observa en la terminal, sino que un error. Esto ocurre por que hemos omitido el símbolo ' en la cadena de texto.

```

1#!/usr/bin/env python3
2# -*- coding: utf-8 -*-
3"""
4Created on Tue Jul  2 14:04:41 2019
5
6@author: mlacarrasco
7"""
8
9 print('este es mi primer texto)

```

File `"/anaconda3/lib/python3.7/site-packages/spyder_kernels/customize/spydercustomize.py"`, line 704, in `runfile`
`execfile(filename, namespace)`

File `"/anaconda3/lib/python3.7/site-packages/spyder_kernels/customize/spydercustomize.py"`, line 108, in `execfile`
`exec(compile(f.read(), filename, 'exec'), namespace)`

File `"/Users/mlacarrasco/Dropbox/_BACKUP/Asignaturas/2019/sem_01/MagisterDataScience/clases/modulo_1_control/code/test.py"`, line 9
`print('este es mi primer texto)`
`^`
SyntaxError: EOL while scanning string literal

In [3]:

In [3]:

Run file Permissions: RW End-of-lines: LF Encoding: UTF-8 Line: 9 Column: 31 Memory: 63 %

2025 © INTRODUCCIÓN A PYTHON. MIGUEL CARRASCO

✗ print('este es mi primer texto)

👍 print('este es mi primer texto')

▼ print()

Para unir dos cadenas de texto, podemos emplear el símbolo +
Es importante notar que une directamente las cadenas sin dejar un espacio en blanco entre éstas

```
1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
3 """
4 Created on Tue Jul  2 14:04:41 2019
5
6 @author: mlacarrasco
7 """
8
9 print('hola buenos'+ 'amigos')
```

INPUT h o l a b u e n o s + a m i g o s

OUTPUT h o l a b u e n o s a m i g o s

▼ print()

Para imprimir más de una cadena de caracteres, o mezclar cadenas de caracteres con números u otro tipo de datos, podemos hacerlo usando print separando los objetos por coma

The screenshot shows the Spyder Python IDE interface. The editor window displays a Python script with the following content:

```
1#!/usr/bin/env python3
2# -*- coding: utf-8 -*-
3"""
4Created on Tue Jul  2 14:04:41 2019
5
6@author: mlacarrasco
7"""
8
9print('hola buenos', 'amigos')
```

A blue callout box points to the string 'amigos' in the print statement, with the text "Cadena de caracteres". Below the editor, the input and output of the script are shown:

INPUT: h o l a b u e n o s , a m i g o s

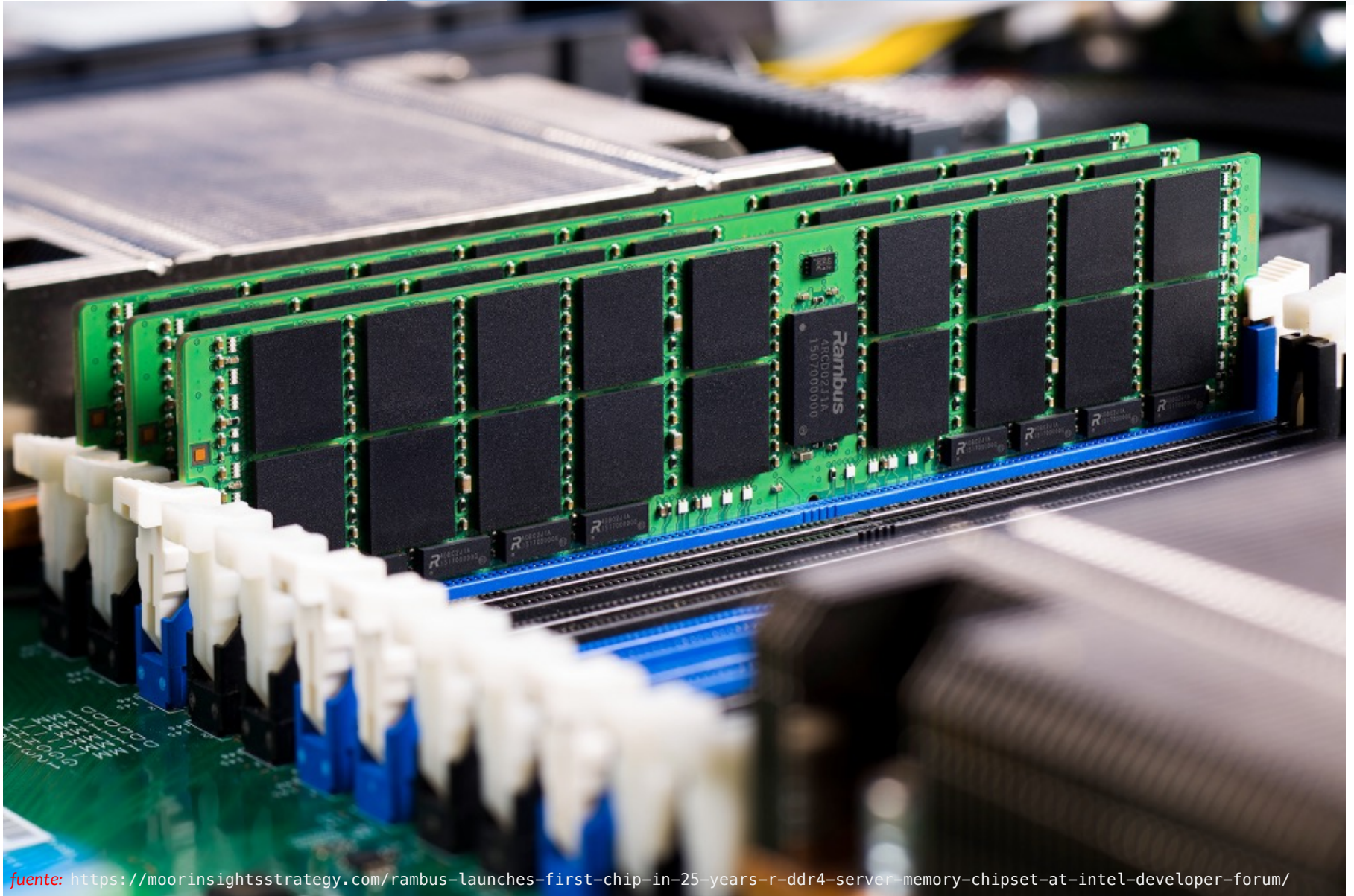
OUTPUT: h o l a b u e n o s a m i g o s

- ▶ Introducción a Python
- ▶ Primeros pasos
 - Lección 01: Despliegue de resultados
 - Lección 02: Variables



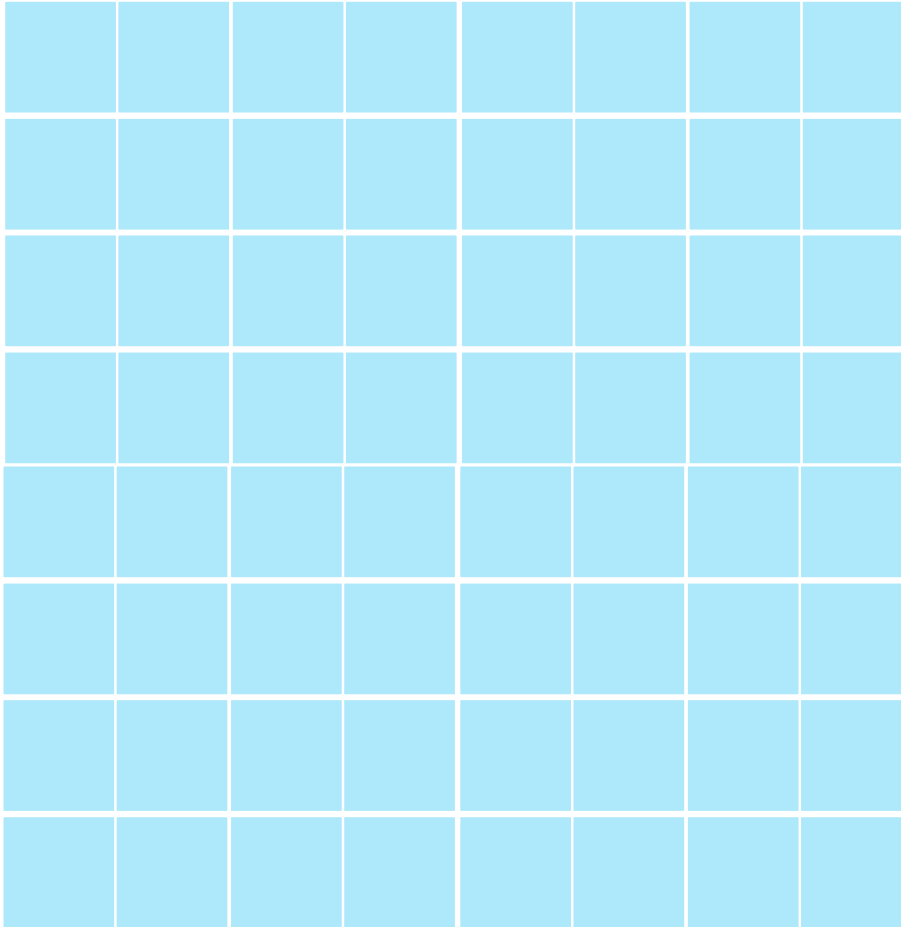
```
self.FidValue = OrderedDict(sorted(self.FidValue.items(), key=lambda item: item[0]))  
#Read item in dictionary  
for key, value in self.FidValue.items():  
    typeOfFID = mapFidType[key]  
    if (typeOfFID == "DATE"):  
        d = datetime.datetime.strptime(str(value), "%Y-%m-%d")  
        dataCal = datetime.date.strptime(str(value), "%Y-%m-%d")  
        FidAndValue = FidAndValue + value  
    else: FidAndValue = FidAndValue + value
```

```
try:  
    start = date(int(self.start_year.get()),  
                self.months.index(self.start_month.get()),  
                int(self.start_day.get()))  
  
    end = date(int(self.end_year.get()),  
              self.months.index(self.end_month.get()),  
              int(self.end_day.get()))
```



fuente: <https://moorinsightsstrategy.com/rambus-launches-first-chip-in-25-years-r-ddr4-server-memory-chipset-at-intel-developer-forum/>

► Creación de variables



vista de bloques en la memoria del computador

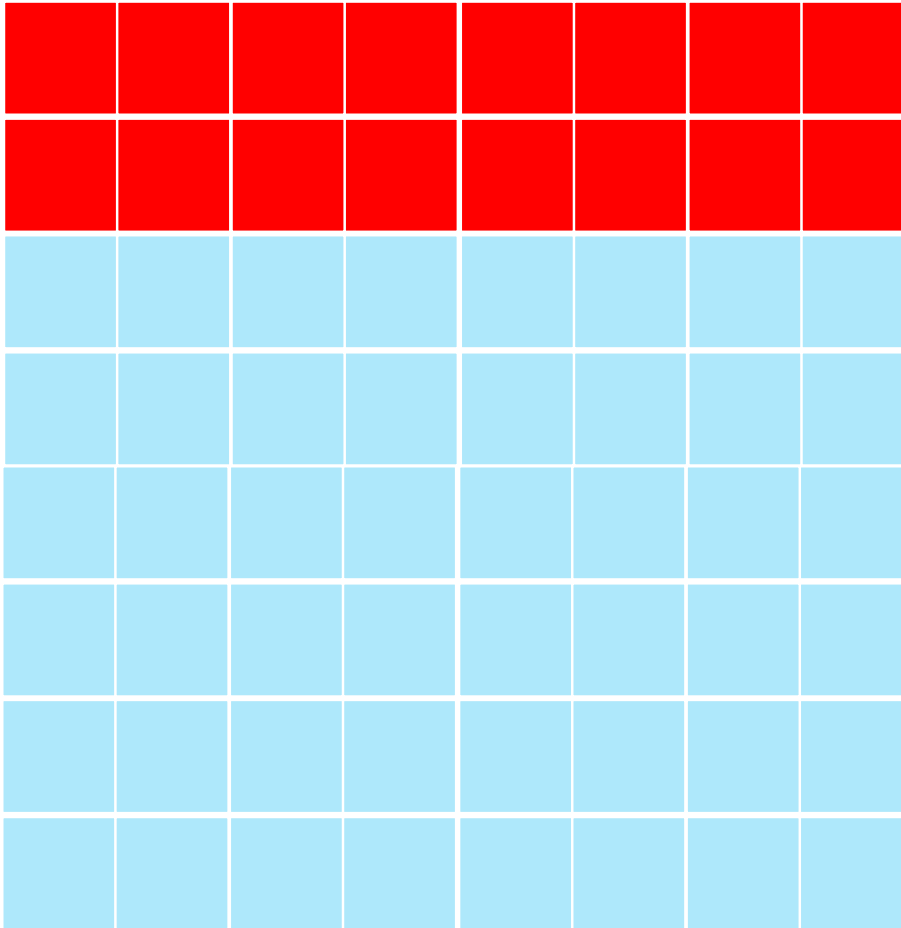
Supongamos que la memoria que tenemos dispone de 64 bloques.

Cada bloque puede guardar un número de 8 bits.

¿Cuántos bytes podemos almacenar?



► Creación de variables



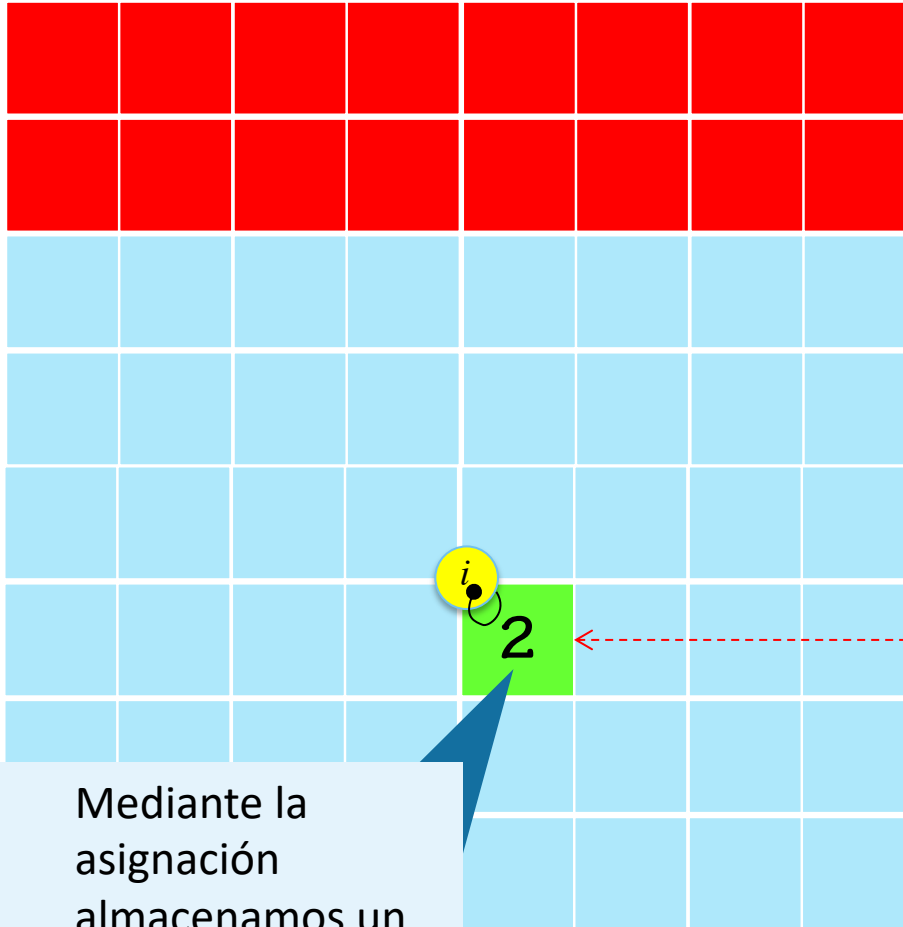
vista de bloques en la memoria del computador

Cuando encendemos el computador, el sistema operativo utiliza parte de la memoria RAM

Lo mismo sucede con los programas que empleamos



► Creación de variables



Mediante la
asignación
almacenamos un
valor en la
memoria

$i = 2$

operador
asignación

! La asignación es
siempre del lado
derecho al izquierdo



- ▶ Introducción a Python
- ▶ Primeros pasos
 - Lección 01: Despliegue de resultados
 - Lección 02: Variables
 - Lección 03: Operaciones aritméticas



```
self.FidValue = OrderedDict(sorted(self.FidValue.items(), key=lambda item: item[0]))
#Read item in dictionary
for key, value in item.FidValue.items():
    typeOfFID = mapFidType[key]
    if (typeOfFID == "DATE"):
        d = datetime.datetime.strptime(str(value), "%Y-%m-%d")
        dataCal = datetime.date.strptime(str(value), "%Y-%m-%d")
        FidAndValue = FidAndValue + value
    else: FidAndValue = FidAndValue + value
```

```
try:
    start = date(int(self.start_year.get()),
                 self.months.index(self.start_month.get()),
                 int(self.start_day.get()))

    end = date(int(self.end_year.get()),
               self.months.index(self.end_month.get()),
               int(self.end_day.get()))
```

suma



resta



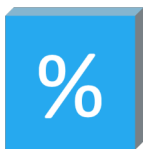
multiplicación



división



módulo



Operaciones aritméticas

Python permite efectuar las siguientes operaciones aritméticas

1

```
w = 100  
i = 2 + w
```

2

```
w = 100  
i = (2 + w)  
i = i - 3
```

3

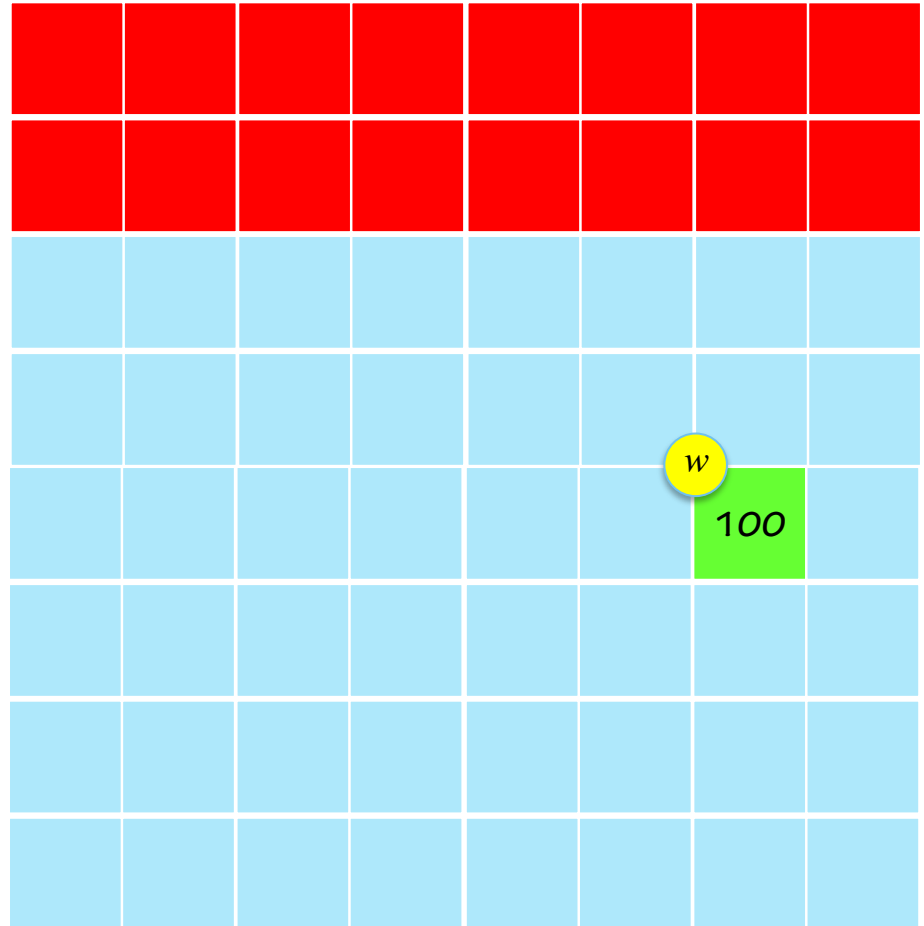
```
w = 100  
i = (w - 98) * (w * 2)
```

4

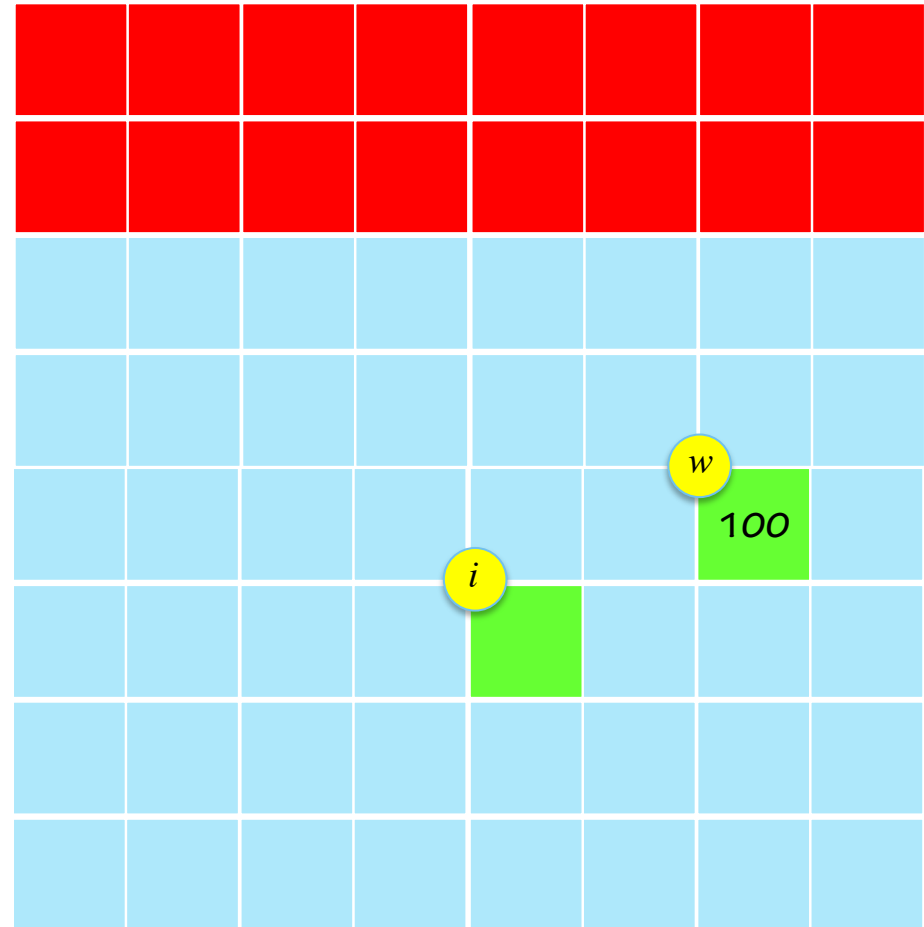
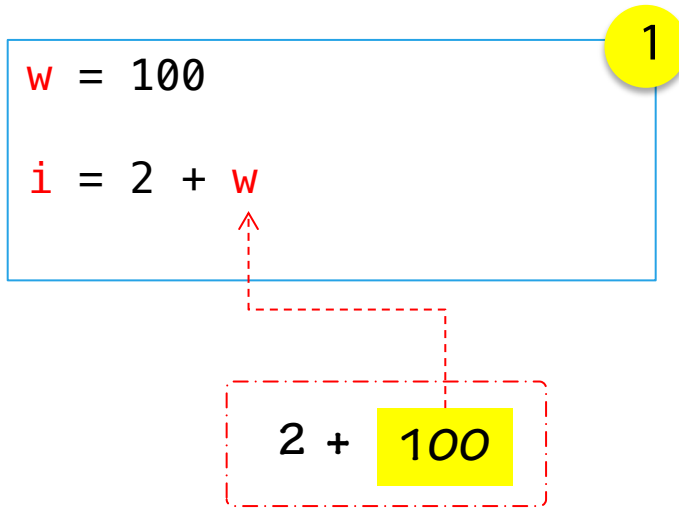
```
w = 100  
i = 2  
w = w / 2  
i = w - i
```

$w = 100$

1

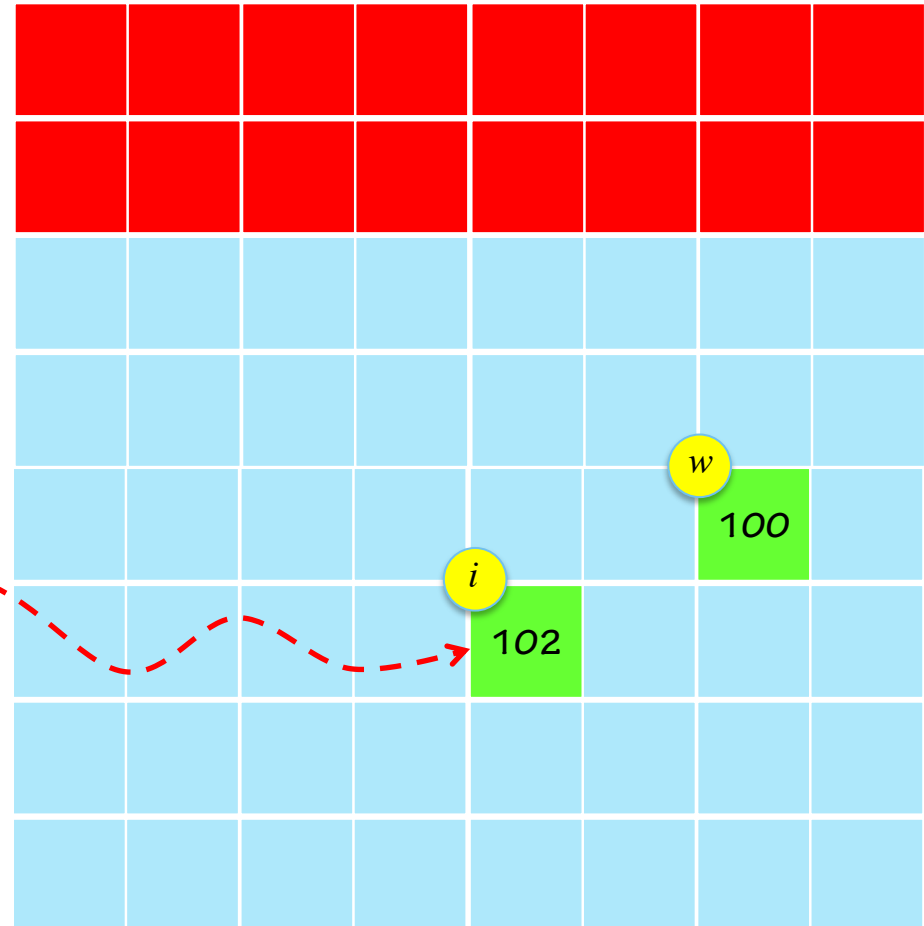
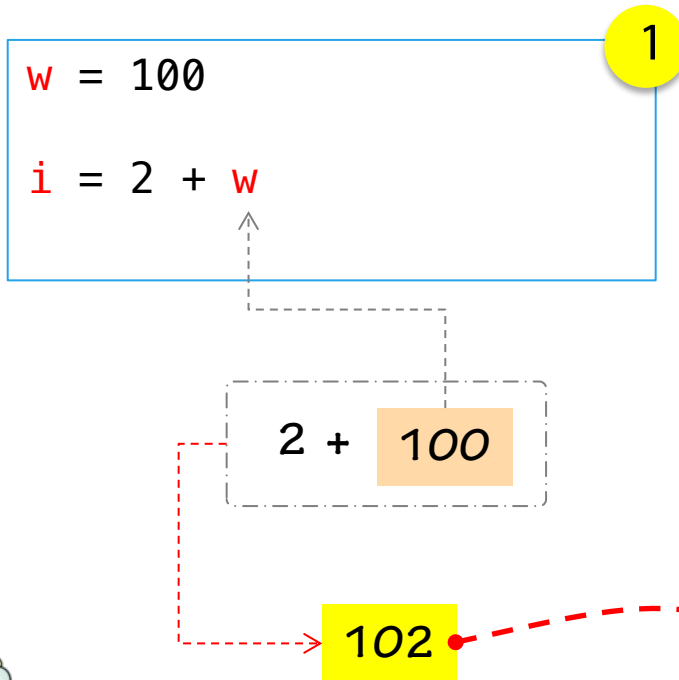


Toda variable tiene un tipo de datos definido.
Si empleamos varias variables del mismo tipo solo debemos separarlas por comas.



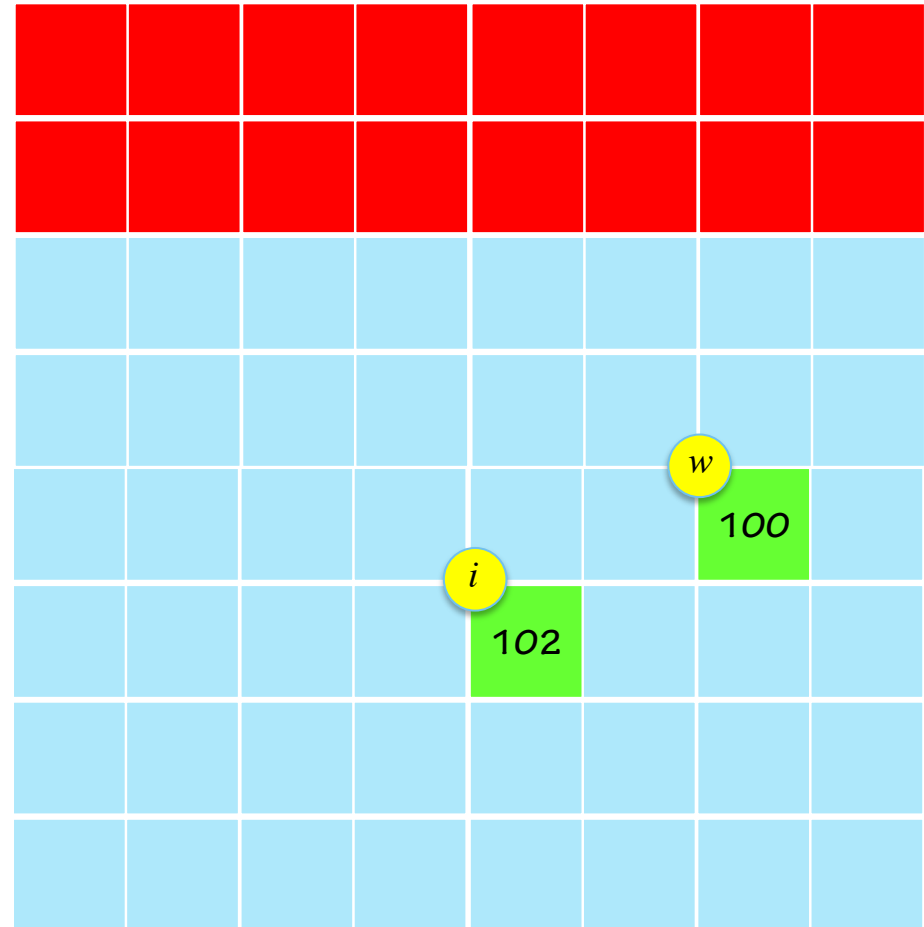
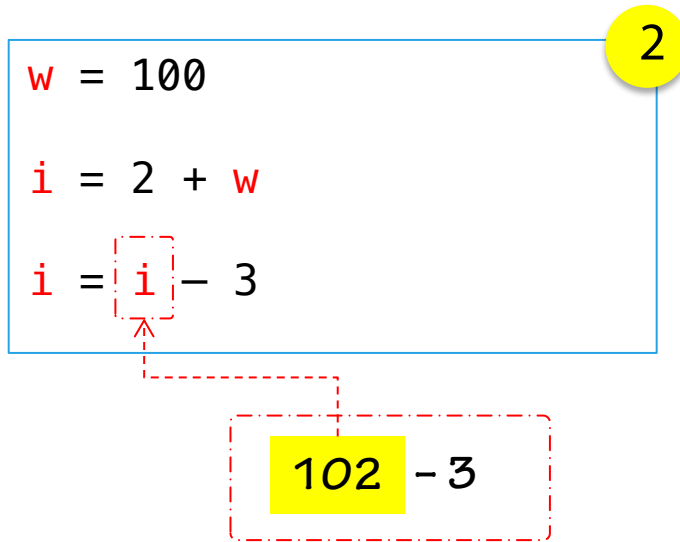
Siempre debe
calcularse (evaluarse)
la parte derecha de la
asignación y luego se
almacena el resultado
al lado izquierdo





Siempre debe
calcularse (evaluarse)
la parte derecha de la
asignación y luego se
almacena el resultado
al lado izquierdo



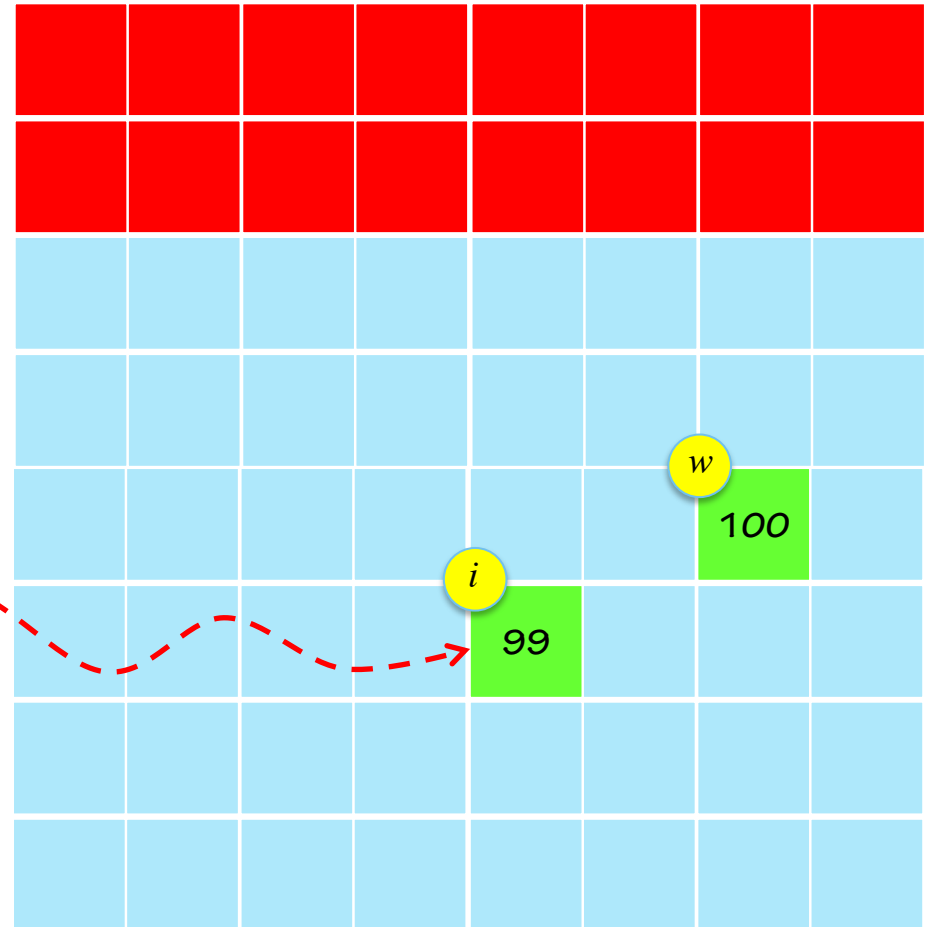


2

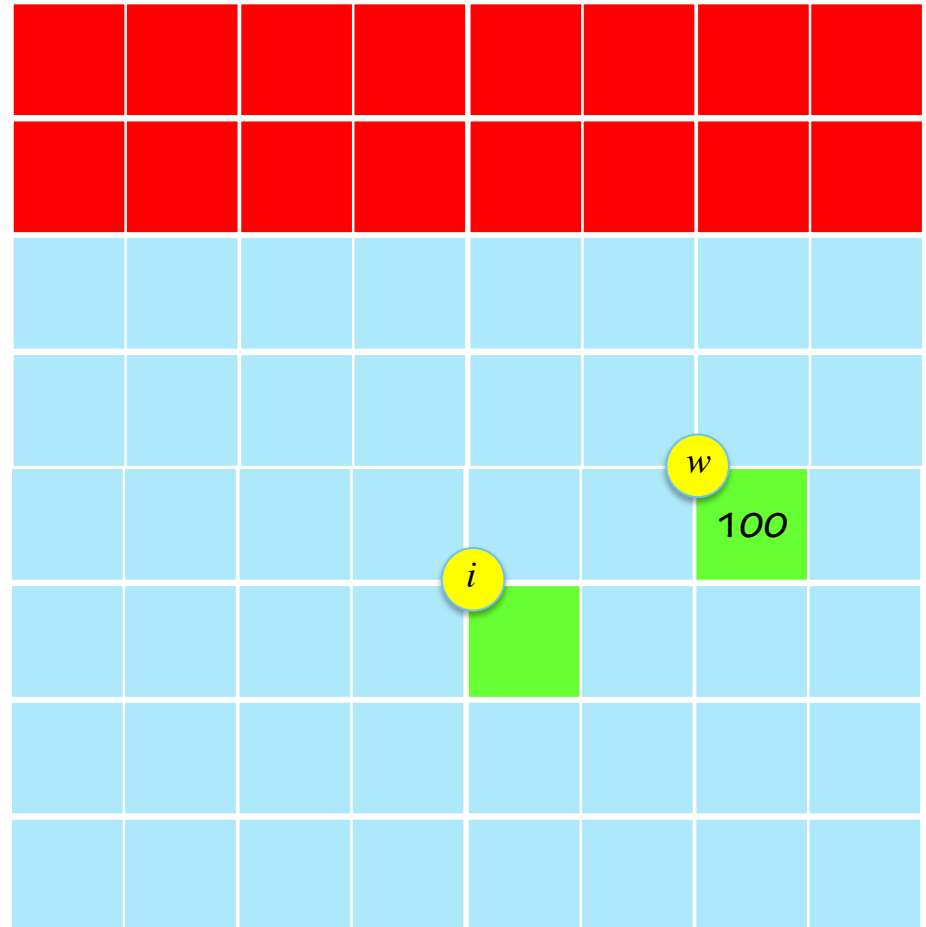
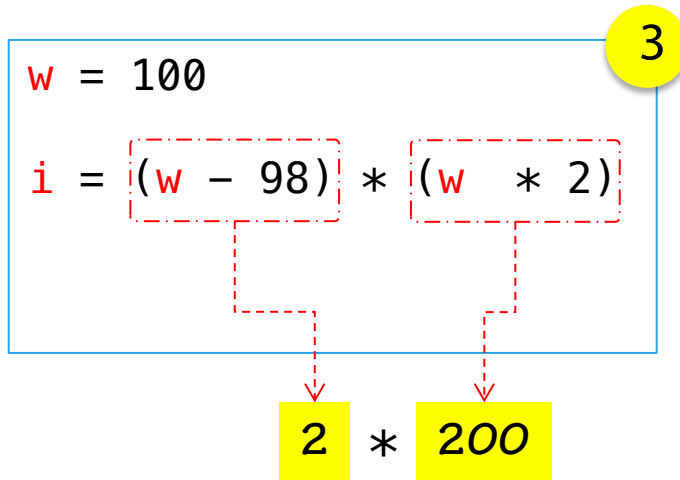
```
w = 100  
i = 2 + w  
i = i - 3
```

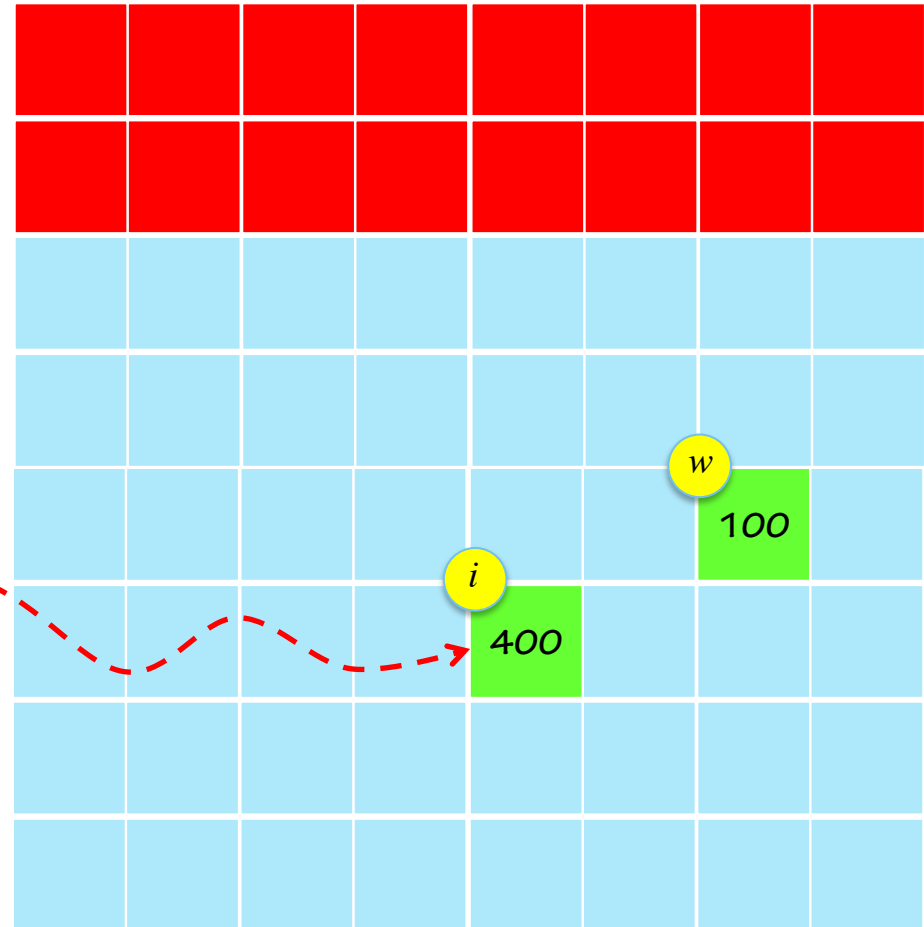
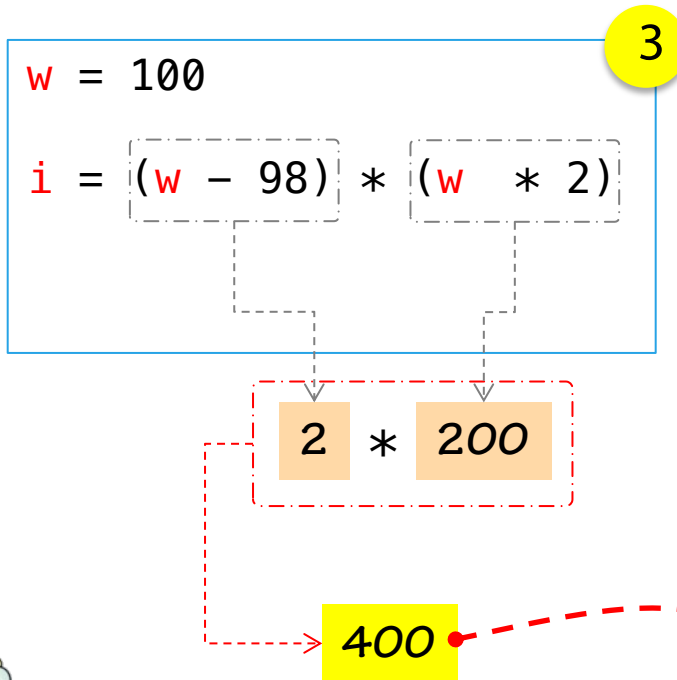
102 - 3

99

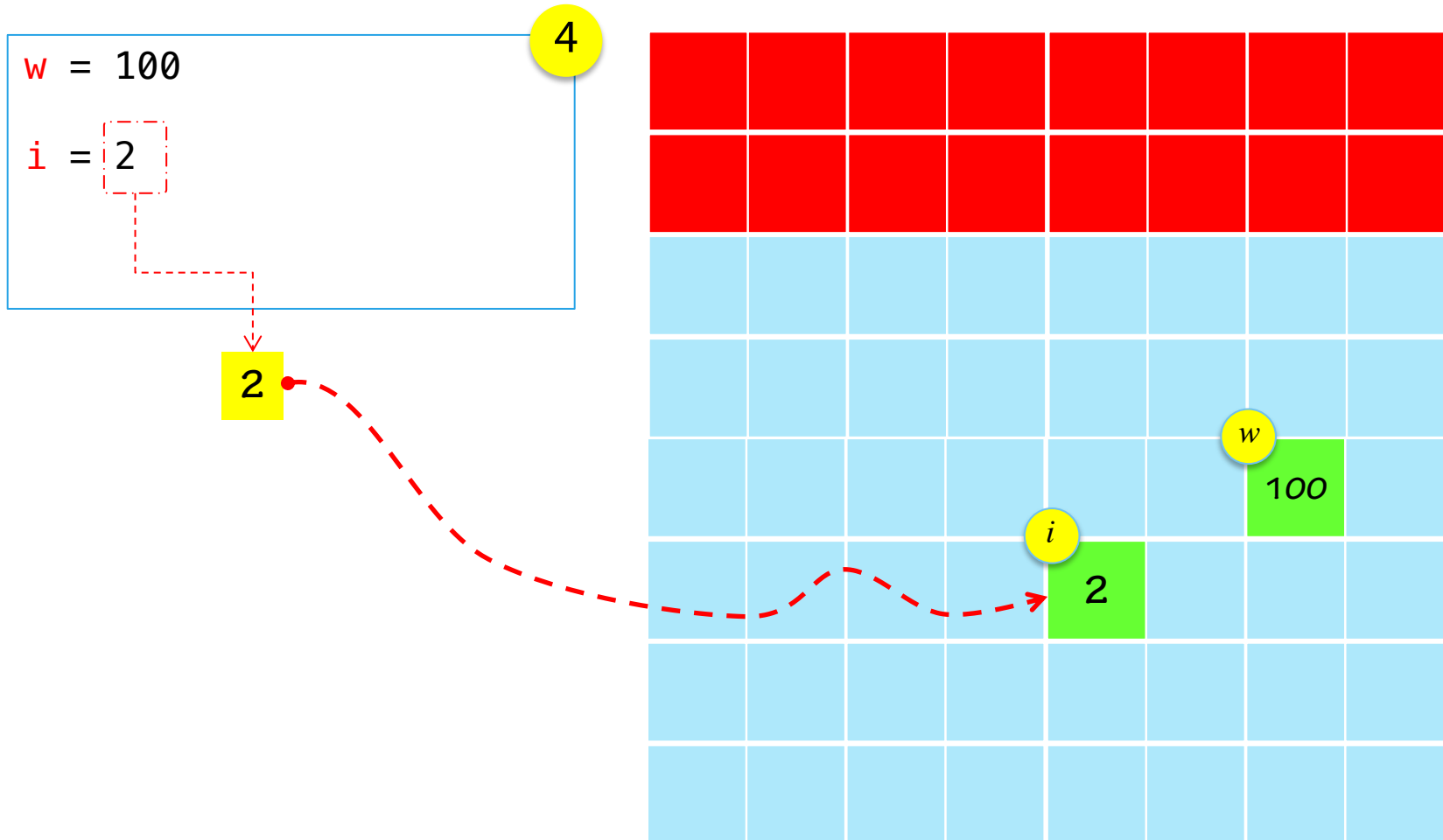


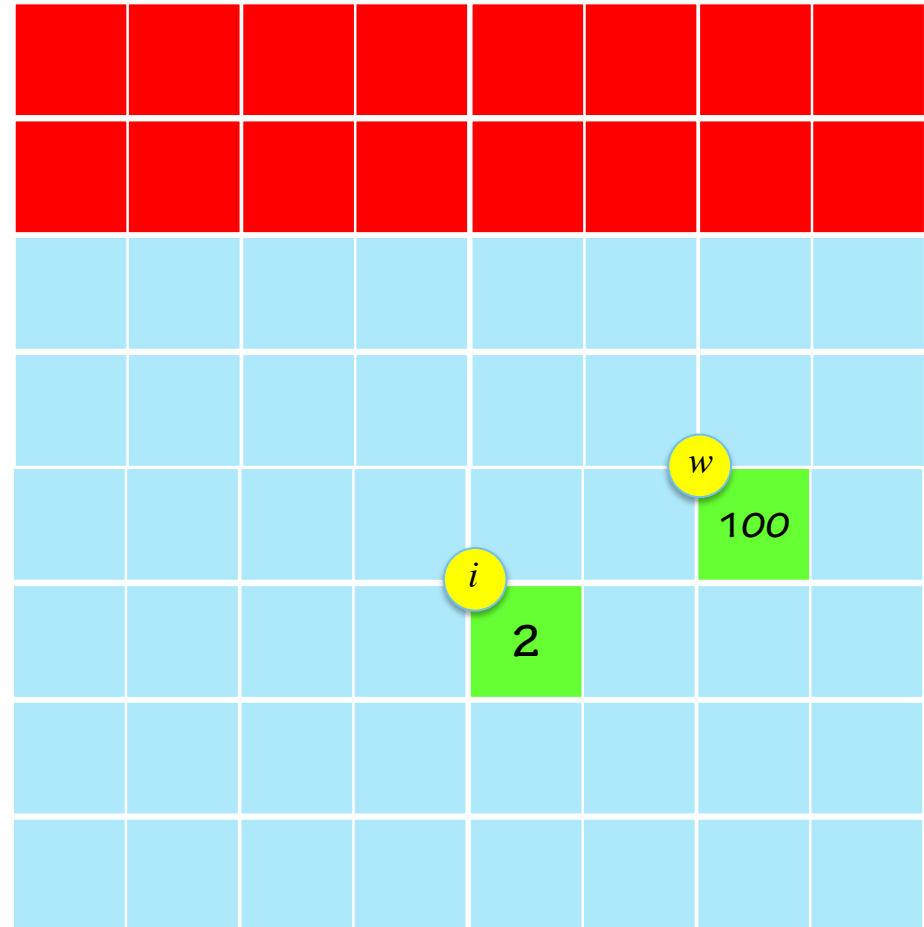
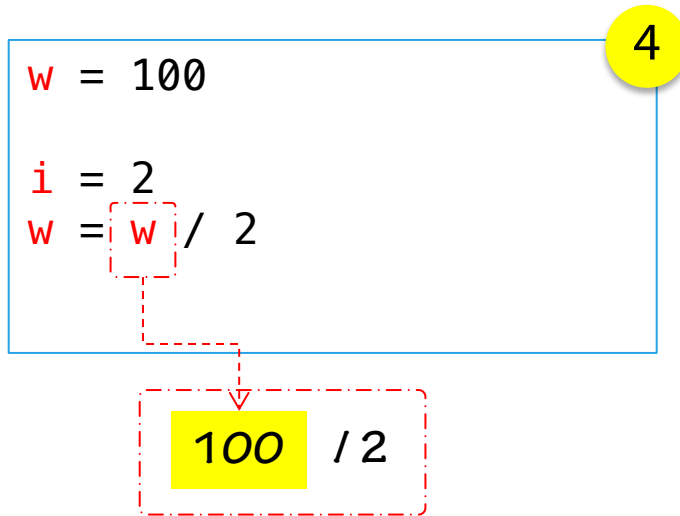
Si observamos la variable *i*, ésta no ha modificado su valor hasta que se haya asignado algo en ella



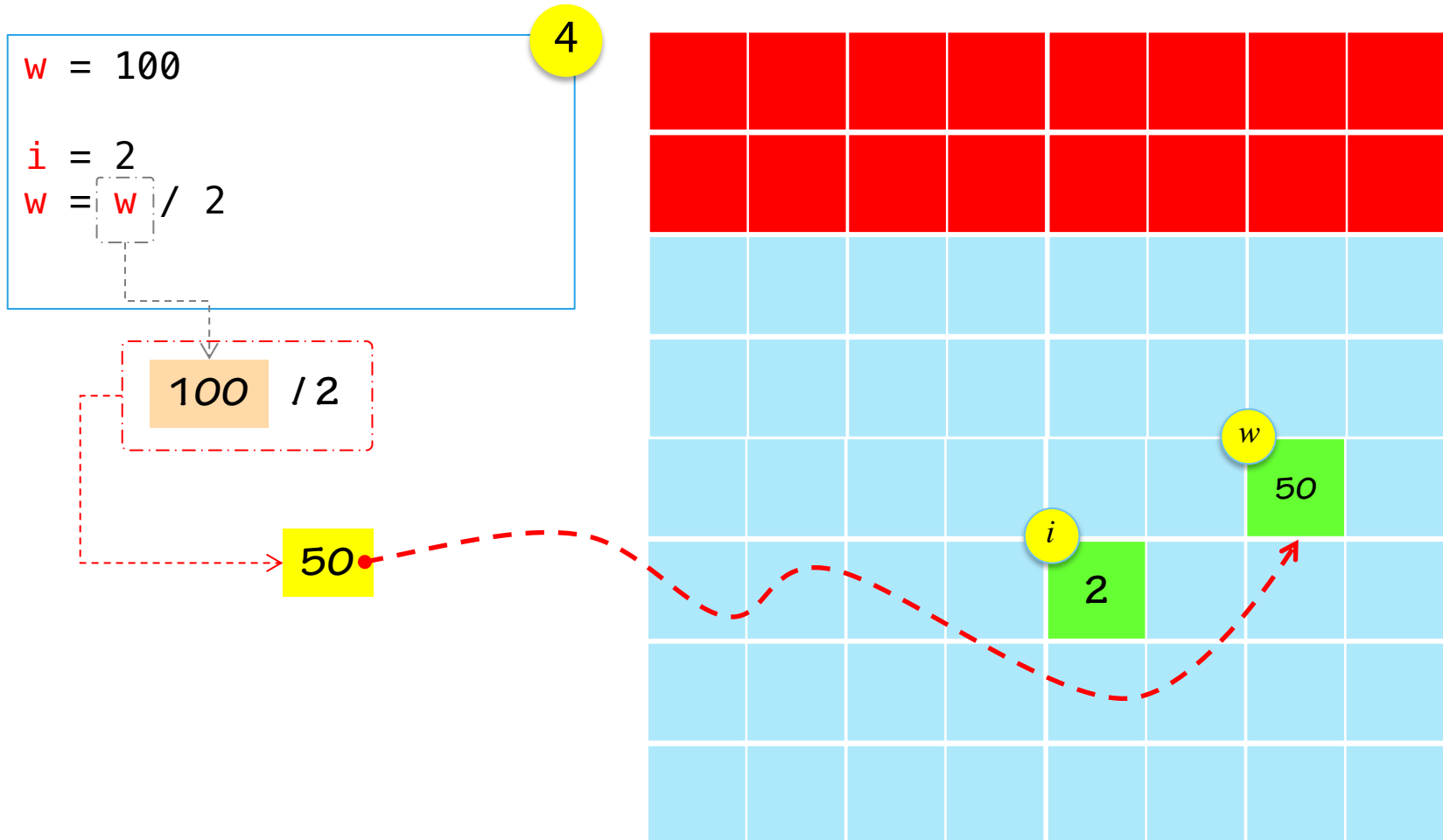


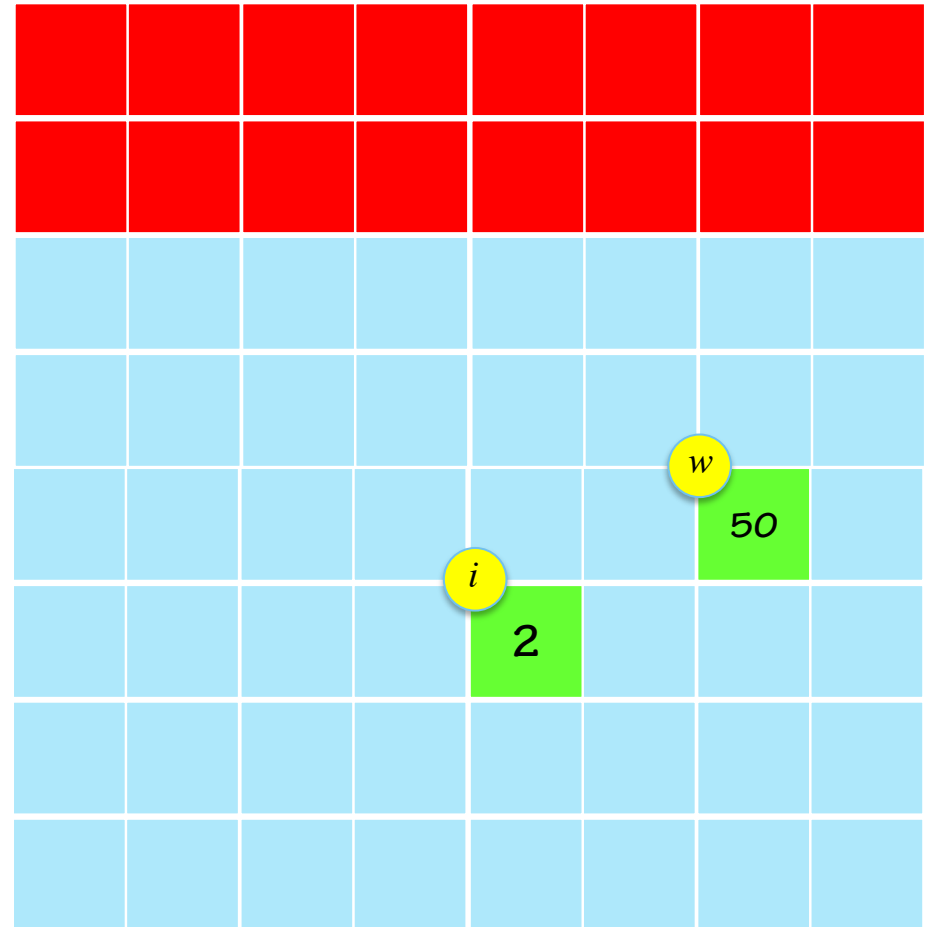
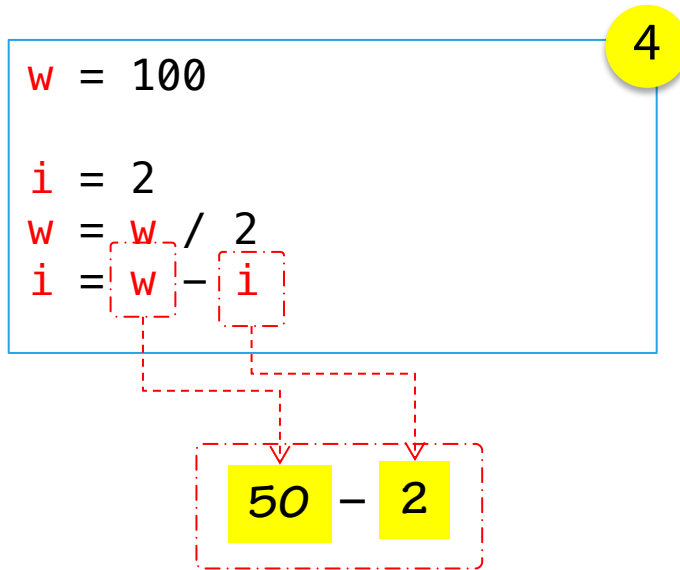
Recuerde, siempre debe terminar de calcular las operaciones y luego se asigna a la variable definida.





Recuerde, siempre debe terminar de calcular las operaciones y luego se asigna a la variable definida.



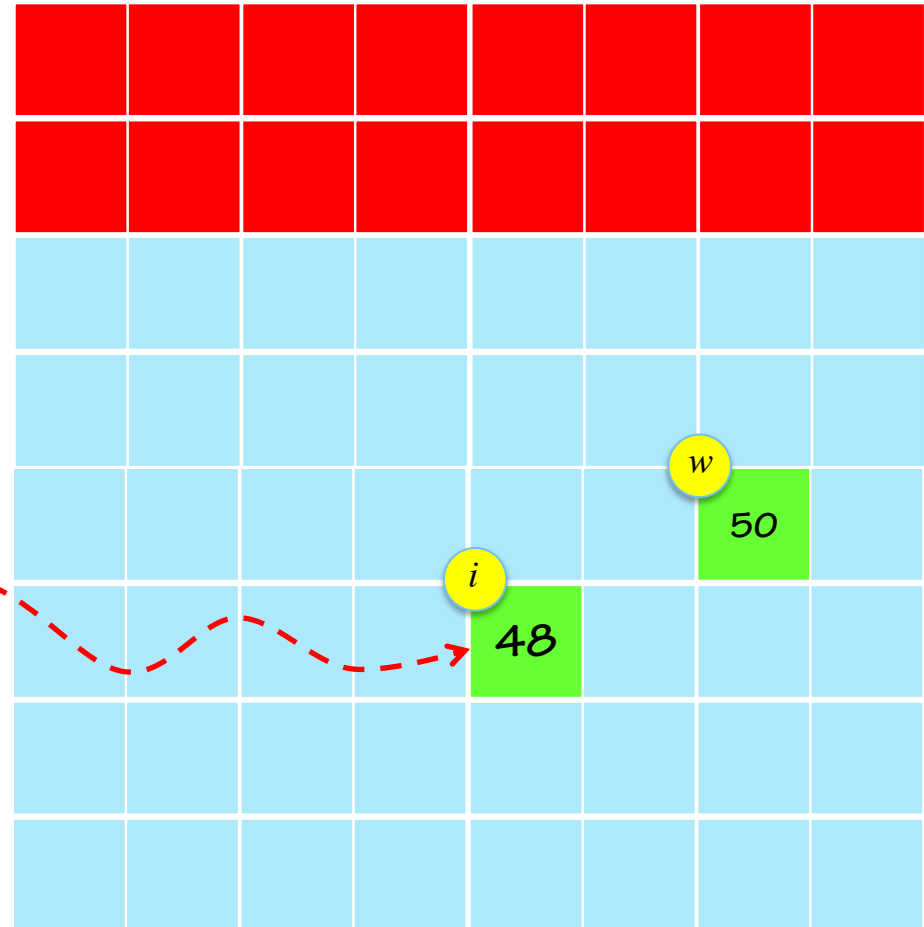


4

$$w = 100$$
$$i = 2$$
$$w = w / 2$$
$$i = w - i$$

$$50 - 2$$

48



Recuerde, siempre debe
terminar de calcular las
operaciones y luego se
asigna a la variable
definida.

suma



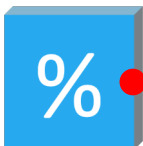
resta



división



módulo



multiplicación



$$a \% b = m \rightarrow a = b \times k + m$$

$$11 = 2 \times 5 + 1$$

$$18 = 2 \times 9 + 0$$

módulo o resto
división

módulo o resto
división

Orden de Precedencia

! Recuerda. El orden de precedencia es importante. Este es el orden que utiliza Python

1^{ro}

paréntesis

()

2^{do}

exponente

**

3^{ro}

izquierda a derecha

multipli
cación

*

división

/

módulo

%

4^{to}

izquierda a derecha

suma

+

resta

-



Recordemos que una variable representa **un espacio de memoria en el computador** que puede ser modificado en el tiempo y permite el registro y acceso a los datos.

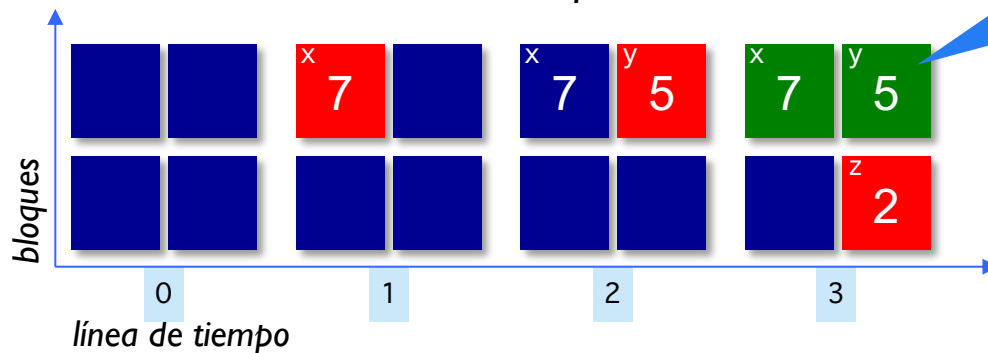
Matemáticas

```
x = 7
y = 5
z = x - y
```

Python

```
• x = 7
y = 5
z = x - y
print(z)
```

memoria del computador



El computador reserva un espacio en memoria y a medida que se ejecutan instrucciones, se acceden y modifican los datos según la ejecución del programa

■ sin cambios
■ registro
■ lectura



Una variable representa **un espacio de memoria en el computador** que puede ser modificado en el tiempo y permite el registro y acceso a los datos.

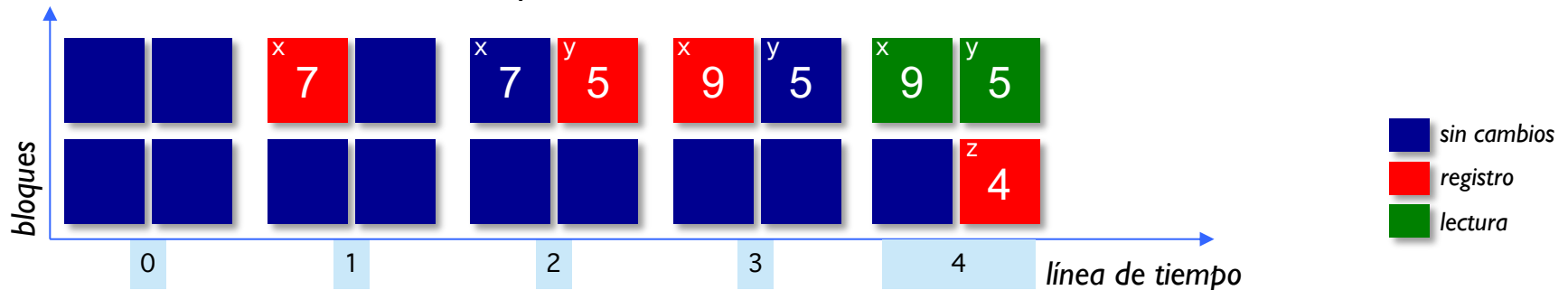
Matemáticas

```
x = 7
y = 5
x = 9
z = x - y
```

Python

```
1 x = 7
2 y = 5
3 x = 9
4 z = x - y
5 print(z)
```

memoria del computador





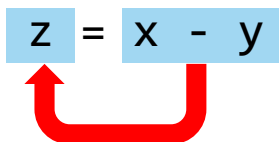
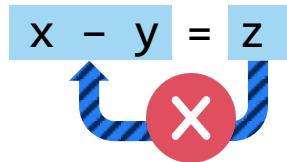
Una variable representa **un espacio de memoria en el computador** que puede ser modificado en el tiempo y permite el registro y acceso a los datos.

Python

```
x = 7
y = 5
x = 9
z = x - y
print(z)
```



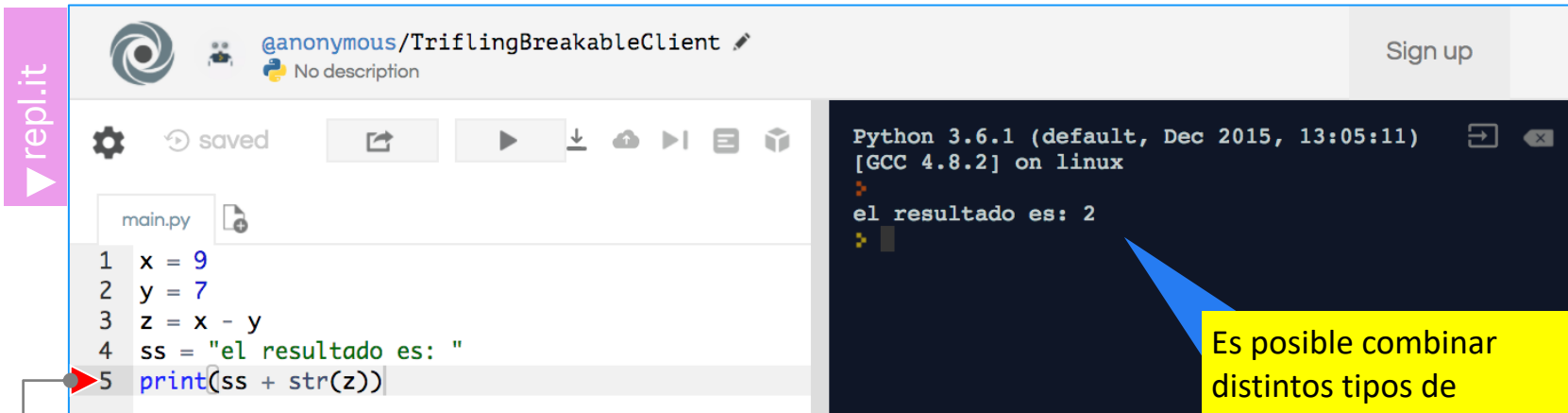
Importante



La asignación es
siempre de
derecha a izquierda

▼ str()

Vimos anteriormente que `print` me permite imprimir varios valores separados por coma. Una alternativa es el comando `str()` que permite transformar un número en una cadena de texto. Con ello podemos concatenar distintos **tipos de datos** (números o texto)



The screenshot shows a Replit environment with a Python script in `main.py`:

```
1 x = 9
2 y = 7
3 z = x - y
4 ss = "el resultado es: "
5 print(ss + str(z))
```

The output of the script is:

```
Python 3.6.1 (default, Dec 2015, 13:05:11)
[GCC 4.8.2] on linux
el resultado es: 2
```

Below the script, the expression `print(ss + str(z))` is broken down into its components:

- `ss` is labeled as *cadena de texto* (string).
- `+` is the concatenation operator.
- `str(z)` is labeled as *cadena de texto* (string), with `z` specifically labeled as *número* (number).

A yellow callout box points to the output in the terminal, stating: "Es posible combinar distintos tipos de operaciones con variables de números y texto para producir resultados más complejos" (It is possible to combine different types of operations with variables of numbers and text to produce more complex results).



Tiempo : 5 minutos

Vamos a comenzar usando Python como una calculadora. Para ello muestra en pantalla una operación matemática y su resultado. Por ejemplo:

El resultado de `2 * 3 + 4 - 12 ** 5` es `-248822`

¿Qué operaciones matemáticas ocupaste?

¿Tuviste algún problema al imprimir en pantalla?



- ▶ Introducción a Python
- ▶ Primeros pasos
 - Lección 01: Despliegue de resultados
 - Lección 02: Variables
 - Lección 03: Operaciones aritméticas
 - Lección 04: Ingreso de datos



```
self.FidValue = OrderedDict(sorted(self.FidValue.items(), key=lambda item: item[0]))
#Read item in dictionary
for key, value in item.FidValue.items():
    typeOfFID = mapFidType.get(key)
    if (typeOfFID == "DATE"):
        d = datetime.datetime.strptime(str(value), "%Y-%m-%d")
        dataCal = datetime.datetime.strptime(str(d), "%Y-%m-%d")
        FidAndValue = FidAndValue + value
    else: FidAndValue = FidAndValue + value
```

```
try:
    start = date(int(self.start_year.get()),
                 self.months.index(self.start_month.get()),
                 int(self.start_day.get()))

    end = date(int(self.end_year.get()),
               self.months.index(self.end_month.get()),
               int(self.end_day.get()))
```

▼ input()

El comando `input()` permite ingresar un numero por la terminal. El programa espera que el usuario escriba un número y presione la tecla "enter"

The image shows a Python REPL interface with a file named `main.py`. The code in the file is:

```
1
2 ss = "el numero ingresado es: "
3 x= input()
4 print(ss + str(x))
```

The terminal output shows the program running and receiving the input `234`:

```
Python 3.6.1 (default, Dec 2015, 13:05:11)
[GCC 4.8.2] on linux
234
el numero ingresado es: 234
```

A callout box points to the input `234` in the terminal, stating: "cualquier valor alfanumérico será procesado como una cadena de caracteres. Es tarea del programador transformar los caracteres numéricos a números".

Below the code, a diagram illustrates the `x = input()` line. The `input()` part is labeled "cadena de texto" (text string), and the `x =` part is labeled "texto" (text).

▼ input()

El comando `input()` permite ingresar un número por la terminal. El programa espera que el usuario escriba un número y presione la tecla "enter"

```
Python 3.6.1 (default, Dec 2015, 13:05:11)
[GCC 4.8.2] on linux

54
Traceback (most recent call last):
  File "python", line 4, in <module>
TypeError: must be str, not int
```

The screenshot shows a Python REPL window with a file named `main.py`. The code in the file is:

```
1
2 ss = "el numero ingresado es: "
3 x= input()
4 y= x+1
5 print(ss + str(y))
```

A red arrow points to line 4, `y= x+1`, indicating the point of failure. The error message on the right states: `TypeError: must be str, not int`.

$y = x + 1$

cadena de texto

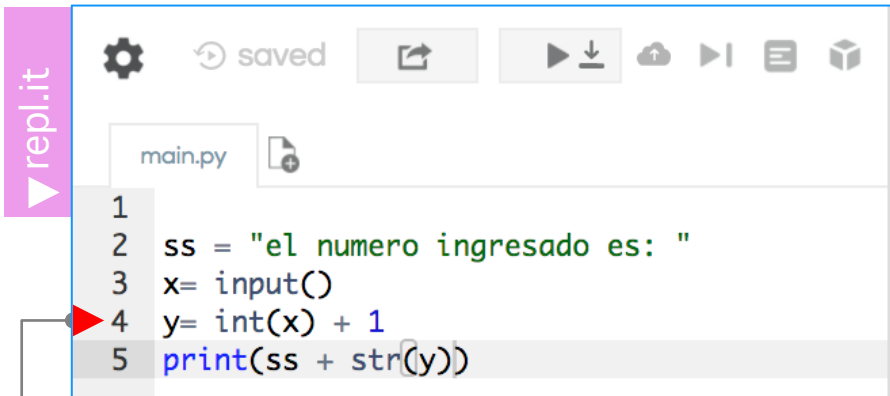
resultado de la operación

número entero

El error se genera por que el intérprete comprende que el valor almacenado en la variable 'x' es una cadena de caracteres y no un valor entero (número)

▼ int()

El comando `int()` transforma una cadena alfanumérica en un número entero. **Importante**: Usted debe ingresar un número entero sin decimales, sin caracteres.



```

1
2 ss = "el numero ingresado es: "
3 x= input()
4 y= int(x) + 1
5 print(ss + str(y))
    
```

Diagram illustrating the data types in the code snippet `y = int(x) + 1`:

- `y`: número entero
- `int(x)`: número entero
- `x`: cadena de texto
- `1`: número entero

Python 3.6.1 (default, Dec 2015, 13:05:11)
[GCC 4.8.2] on linux

```

54
el numero ingresado es: 55
    
```

El proceso anterior se denomina conversión de tipos. Consiste en cambiar el tipo de datos de un origen a uno de destino según el formato seleccionado.

▼ float()

El comando `float()` transforma una cadena alfanumérica en un número flotante. **Importante:** Usted puede ingresar un número con decimales (o punto flotante) pero sin caracteres

The image shows a Python REPL interface with a code editor on the left and a terminal on the right. The code in the editor is:

```
1  
2 ss = "el resultado es: "  
3 x= input()  
4 y= float(x) + 1  
5 print(ss + str(y))
```

A red arrow points to line 4, `y= float(x) + 1`. Below this line, a diagram shows the components of the expression:

- `y =` is labeled with a vertical line pointing to it.
- `float(x)` is underlined, with a vertical line pointing to the label "número decimal" (decimal number).
- `+` is labeled with a vertical line pointing to it.
- `1` is underlined, with a vertical line pointing to the label "número decimal" (decimal number).

The terminal on the right shows the execution of the code:

```
Python 3.6.1 (default, Dec 2015, 13:05:11) [GCC 4.8.2] on linux  
1.2  
el resultado es: 2.2
```

A blue callout box points to the terminal output with the text:

El proceso anterior se denomina conversión de tipos. Consiste en cambiar el tipo de datos de un origen a uno de destino según el formato seleccionado.

- ▶ Introducción a Python
- ▶ Primeros pasos
 - Lección 01: Despliegue de resultados
 - Lección 02: Variables
 - Lección 03: Operaciones aritméticas
 - Lección 04: Ingreso de datos
 - Ejercicio 1



```
self.FidValue = OrderedDict(sorted(self.FidValue.items(), key=lambda item: item[0]))
#Read item in dictionary
for key, value in item.FidValue.items():
    typeOfFID = mapFidType[key]
    if (typeOfFID == "DATE"):
        d = datetime.datetime.strptime(str(value), "%Y-%m-%d")
        dataCal = datetime.datetime.strptime(str(self.start_date), "%Y-%m-%d")
        FidAndValue = FidAndValue + value
    else: FidAndValue = FidAndValue + value
```

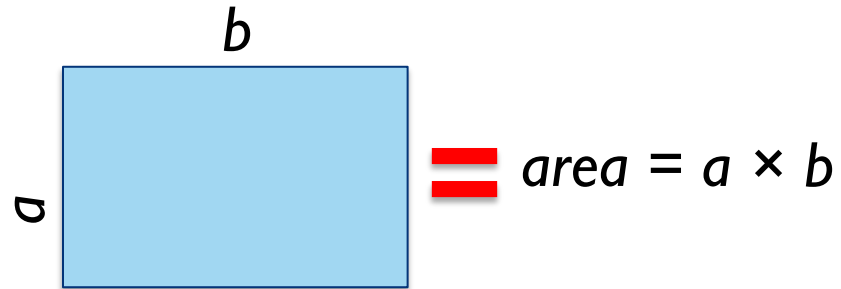
```
try:
    start = date(int(self.start_year.get()),
                 self.months.index(self.start_month.get()),
                 int(self.start_day.get()))

    end = date(int(self.end_year.get()),
               self.months.index(self.end_month.get()),
               int(self.end_day.get()))
```



Tiempo : 5 minutos

Realice un programa que calcule el área de un rectángulo.



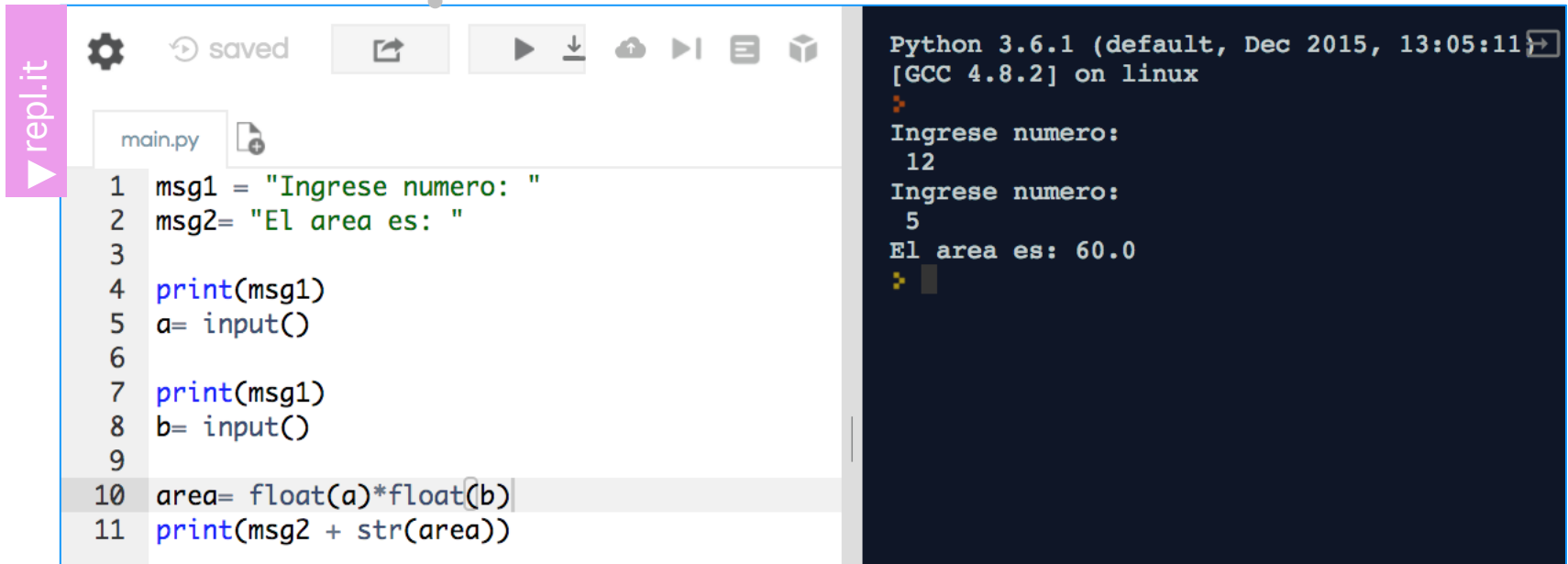
▼ Preguntas

- *¿Cómo se calcula el área?*
- *¿Cómo solicito los datos al usuario?*
- *¿Cómo realizo la multiplicación?*
- *¿Los valores son números enteros o decimales?*
- *¿El resultado es un entero o decimal?*
- *¿Cómo se muestra el resultado?*

Proceso de pensamiento guiado con preguntas, también conocido como pensamiento crítico

Realice un programa que calcule el área de un rectángulo.

- ¿Cómo se calcula el área?→ $\text{area} = a \times b$
- ¿Cómo solicito los datos al usuario?→ `input()`
- ¿Cómo realizo la multiplicación?→ `*`
- ¿Los valores son números enteros o decimales?→ `float()`
- ¿Donde almaceno el resultado?→ `area`
- ¿Cómo se muestra el resultado?→ `print()` y `str()`



The screenshot shows a Replit Python 3.6.1 environment. On the left, the code editor displays a file named `main.py` with the following Python code:

```
1 msg1 = "Ingrese numero: "  
2 msg2= "El area es: "  
3  
4 print(msg1)  
5 a= input()  
6  
7 print(msg1)  
8 b= input()  
9  
10 area= float(a)*float(b)  
11 print(msg2 + str(area))
```

On the right, the terminal window shows the execution of the program. It prompts the user to enter a number twice, with inputs of 12 and 5, and then displays the calculated area: 60.0.

```
Python 3.6.1 (default, Dec 2015, 13:05:11) [GCC 4.8.2] on linux  
Ingrese numero:  
12  
Ingrese numero:  
5  
El area es: 60.0
```

- ▶ Introducción a Python
- ▶ Primeros pasos
- ▶ Estructuras de control
 - Operadores lógicos
 - Condicionales IF-ELSE



```
self.FidValue = OrderedDict(sorted(self.FidValue.items(), key=lambda item: item[0]))
#Read item in dictionary
for key, value in item.FidValue.items():
    typeOfFID = mapFidType[key]
    if (typeOfFID == "DATE"):
        d = datetime.datetime.strptime(str(value), "%Y-%m-%d")
        dataCal = datetime.date.strptime(str(value), "%Y-%m-%d")
        FidAndValue = FidAndValue + str(key) + str(d) + str(dataCal)
    else: FidAndValue = FidAndValue + str(key) + str(value)
```

```
try:
    start = date(int(self.start_year.get()),
                 self.months.index(self.start_month.get()),
                 int(self.start_day.get()))

    end = date(int(self.end_year.get()),
               self.months.index(self.end_month.get()),
               int(self.end_day.get()))
```

comparación



distinto de



mayor



menor



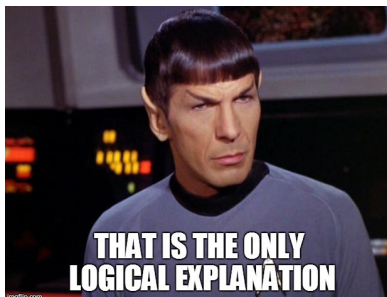
mayor igual



menor igual



Resultados posibles



verdadero
(TRUE)

1

falso
(FALSE)

0

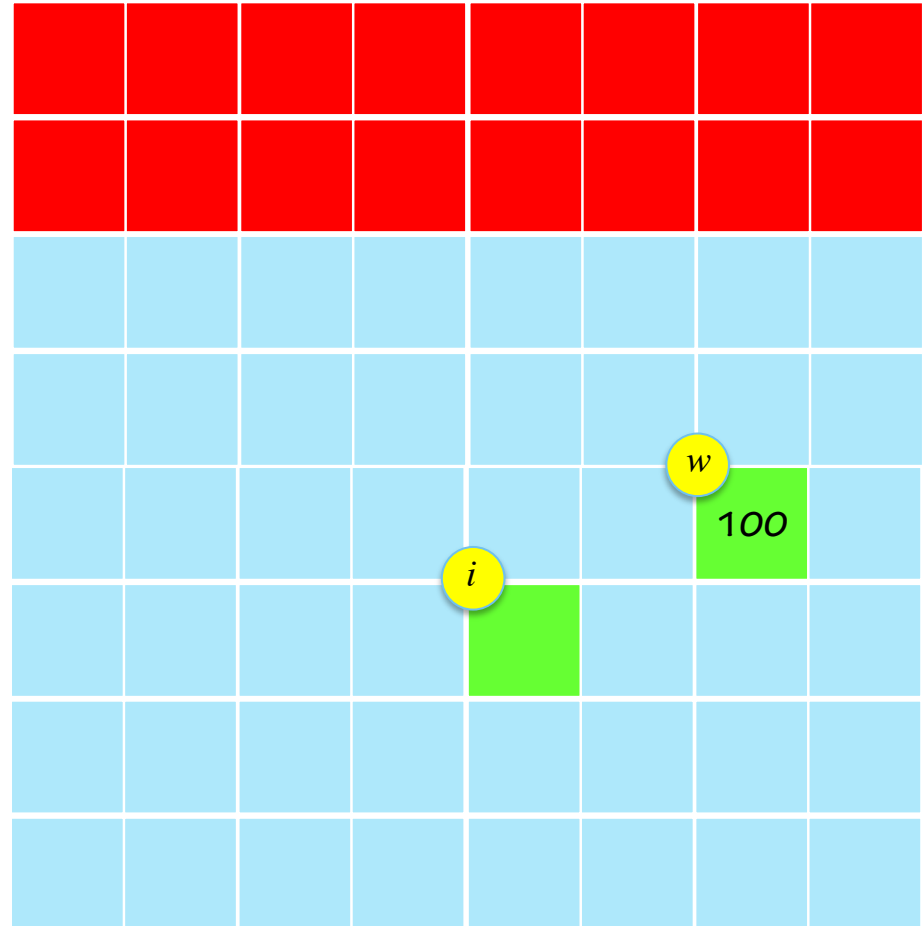
$w = 100$

$i = 99 == w$

$99 == 100$



En el caso de las operaciones relacionales, el único resultado posible es 0 ó 1.



$w = 100$

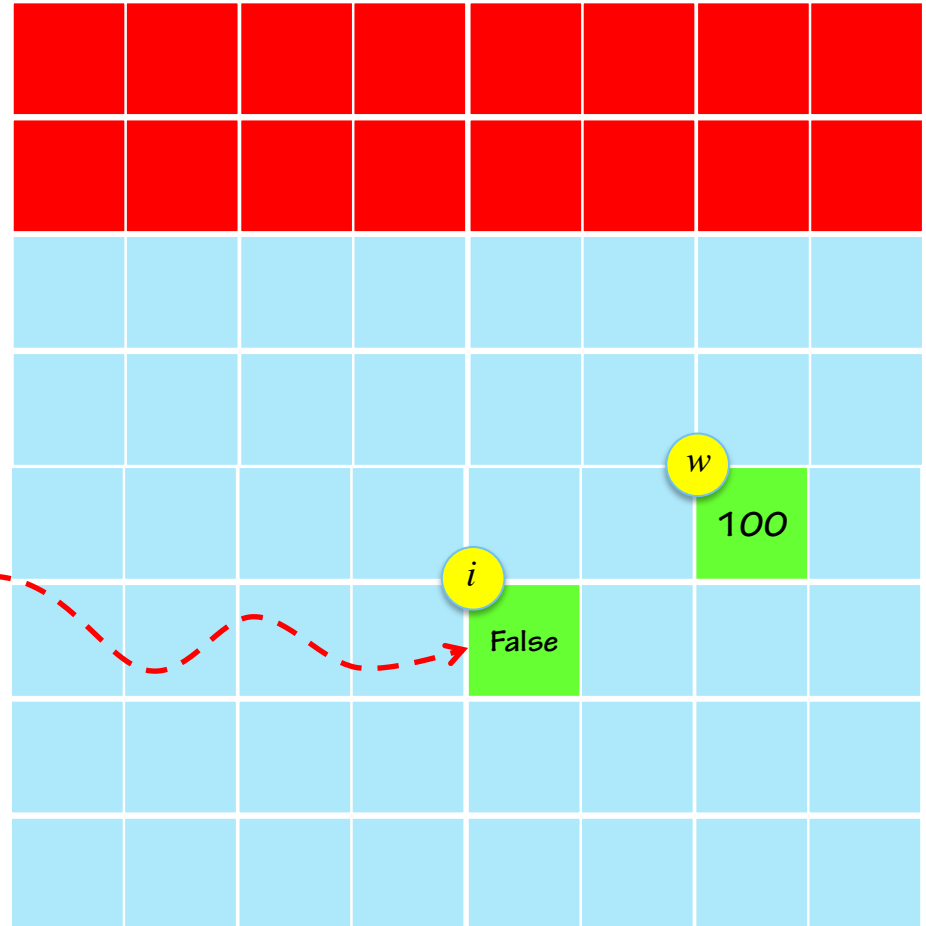
$i = 99 == w$

$99 == 100$

False



⚠ En el caso de las operaciones relacionales, el único resultado posible es False o True



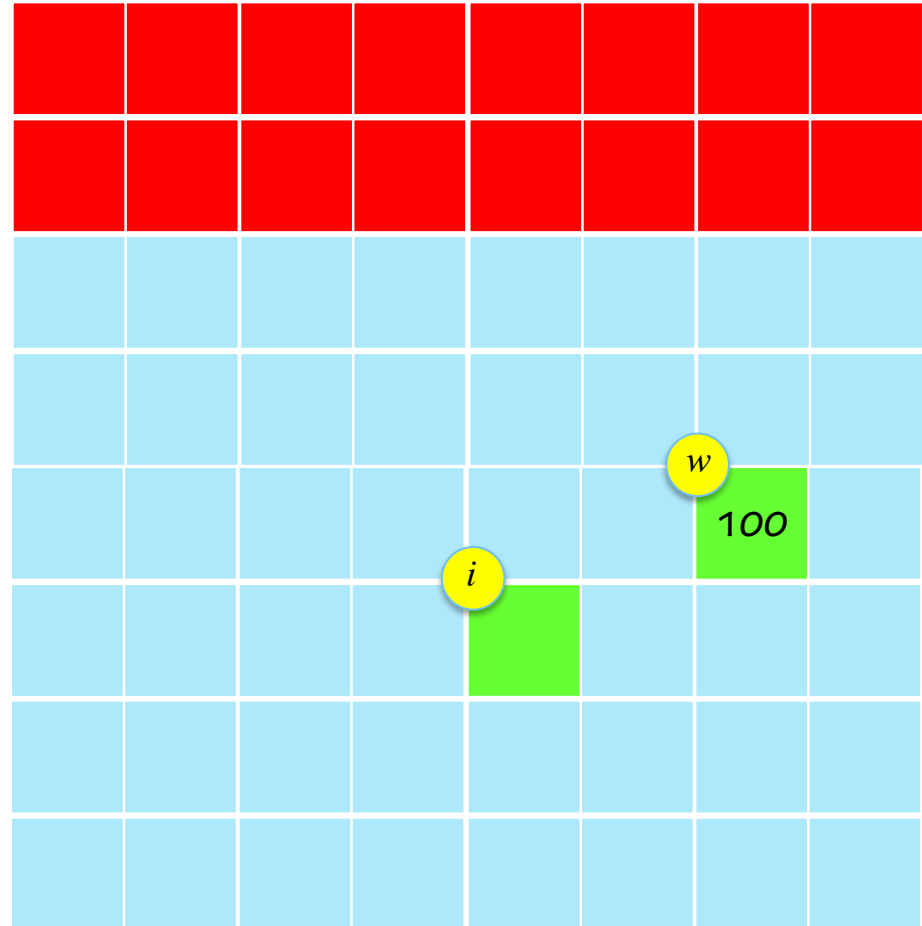
$w = 100$

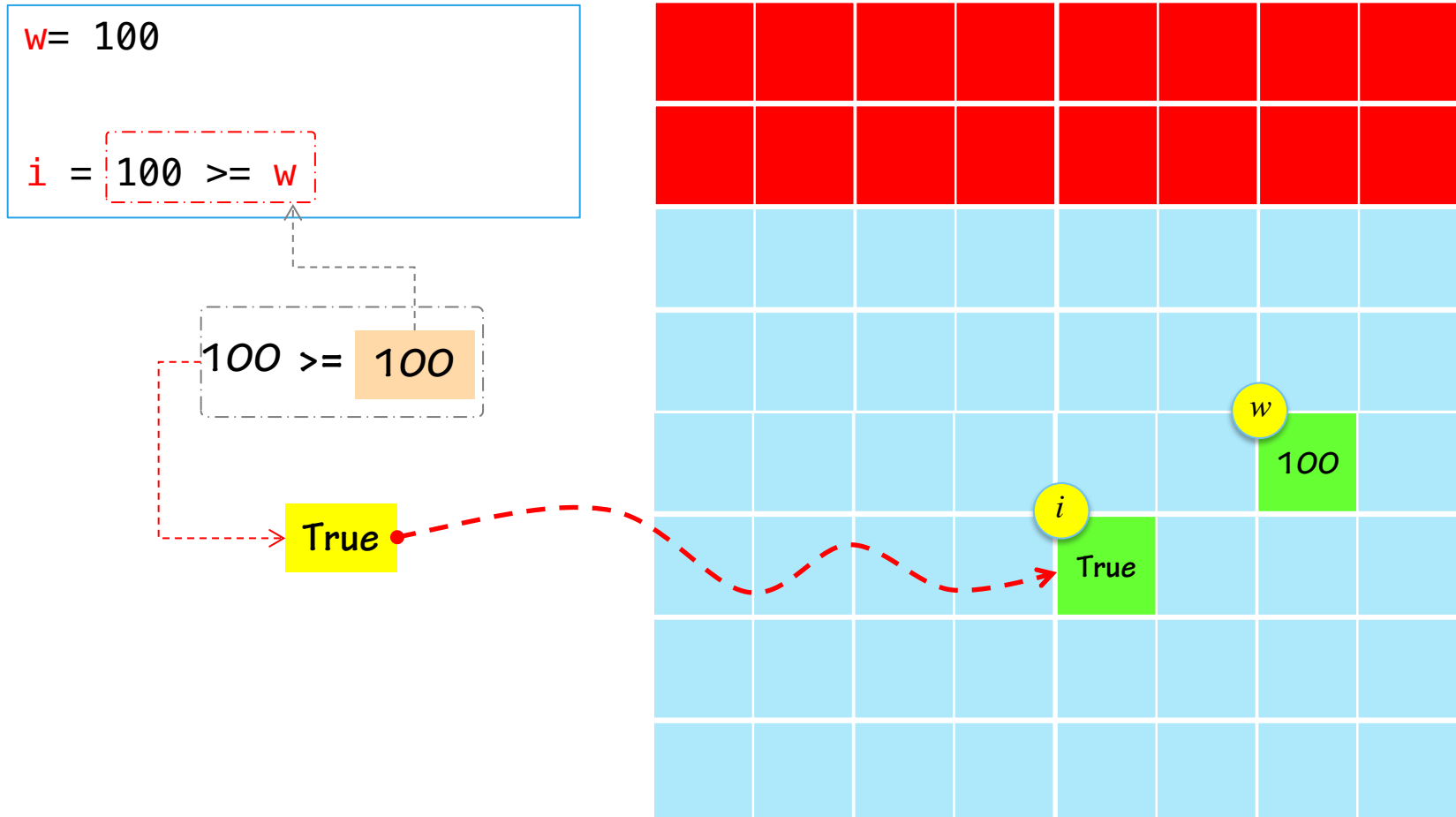
$i = 100 \geq w$

$100 \geq 100$



En el caso de las operaciones relacionales, el único resultado posible es True o False.





operando		Tabla de verdad		
a	b	not(a)	a and b	a or b
True	True	False	True	True
True	False	False	False	True
False	True	True	False	True
False	False	True	False	False

precedencia				
1 ^{ro}	2 ^{do}	3 ^{ro}	4 ^{to}	
==	!=	not	and	or

Operadores lógicos

and

Si **ambos** operandos son verdaderos, la condición se hace verdadera

or

Si **cualquiera** de los operandos es verdadero, la condición se hace verdadera

not

Revierte el resultado lógico de su operando. Si la condición es **verdadera**, se revierte a **falso**

precedencia

1 ^{ro}	2 ^{do}	3 ^{ro}	4 ^{to}
<code>==</code>	<code>!=</code>	<code>not</code>	<code>and</code> <code>or</code>

operando

operando

Tabla de

verdad

a	b	not(a)	a and b	a or b
True	True	False	True	True
True	False	False	False	True
False	True	True	False	True
False	False	True	False	False

Ejemplo

Expresión	Evaluación	Resultado
<code>(25>1) and (10>2)</code>		
<code>not(25>1) and (10>2)</code>		
<code>(-8<-3) or 100<99</code>		
<code>not(25>1) and (-8<-3) or 100<99</code>		

precedencia

1 ^{ro}	2 ^{do}	3 ^{ro}	4 ^{to}
<code>==</code>	<code>!=</code>	<code>not</code>	<code>and</code> <code>or</code>

operando

operando

Tabla de verdad

a	b	not(a)	a and b	a or b
True	True	False	True	True
True	False	False	False	True
False	True	True	False	True
False	False	True	False	False

Ejemplo

Expresión	Evaluación	Resultado
<code>(25>1) and (10>2)</code>	True and True	True
<code>not(25>1) and (10>2)</code>		
<code>(-8<-3) or 100<99</code>		
<code>not(25>1) and (-8<-3) or 100<99</code>		

precedencia

1 ^{ro}	2 ^{do}	3 ^{ro}	4 ^{to}
<code>==</code>	<code>!=</code>	<code>not</code>	<code>and</code> <code>or</code>

operando

operando

Tabla de verdad

a	b	not(a)	a and b	a or b
True	True	False	True	True
True	False	False	False	True
False	True	True	False	True
False	False	True	False	False

Ejemplo

Expresión	Evaluación	Resultado
<code>(25>1) and (10>2)</code>	True and True	True
<code>not(25>1) and (10>2)</code>	False and True	False
<code>(-8<-3) or 100<99</code>		
<code>not(25>1) and (-8<-3) or 100<99</code>		

precedencia

1 ^{ro}	2 ^{do}	3 ^{ro}	4 ^{to}
<code>==</code>	<code>!=</code>	<code>not</code>	<code>and</code> <code>or</code>

operando

operando

Tabla de verdad

a	b	not(a)	a and b	a or b
True	True	False	True	True
True	False	False	False	True
False	True	True	False	True
False	False	True	False	False

Ejemplo

Expresión	Evaluación	Resultado
<code>(25>1) and (10>2)</code>	True and True	True
<code>not(25>1) and (10>2)</code>	False and True	False
<code>(-8<-3) or 100<99</code>	True or False	True
<code>not(25>1) and (-8<-3) or 100<99</code>		

precedencia

1 ^{ro}	2 ^{do}	3 ^{ro}	4 ^{to}
<code>==</code>	<code>!=</code>	<code>not</code>	<code>and</code> <code>or</code>

operando

operando

Tabla de verdad

a	b	not(a)	a and b	a or b
True	True	False	True	True
True	False	False	False	True
False	True	True	False	True
False	False	True	False	False

Ejemplo

Expresión	Evaluación	Resultado
<code>(25>1) and (10>2)</code>	True and True	True
<code>not(25>1) and (10>2)</code>	False and True	False
<code>(-8<-3) or 100<99</code>	True or False	True
<code>not(25>1) and (-8<-3) or 100<99</code>	(False and True) or False = False or False	False

- 

```

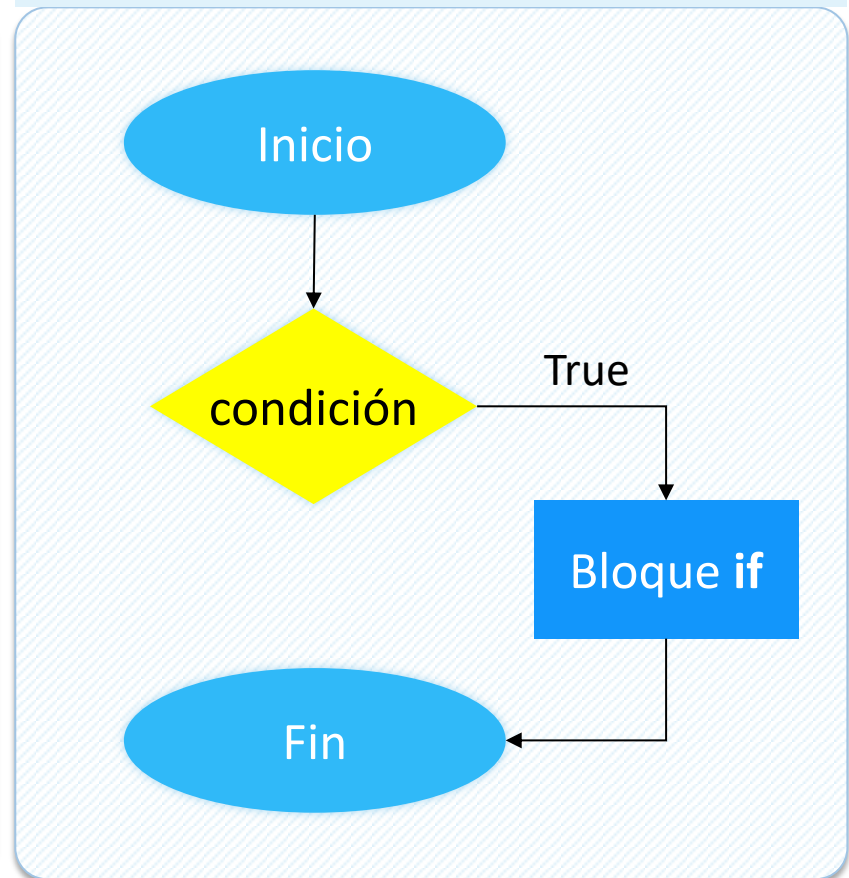
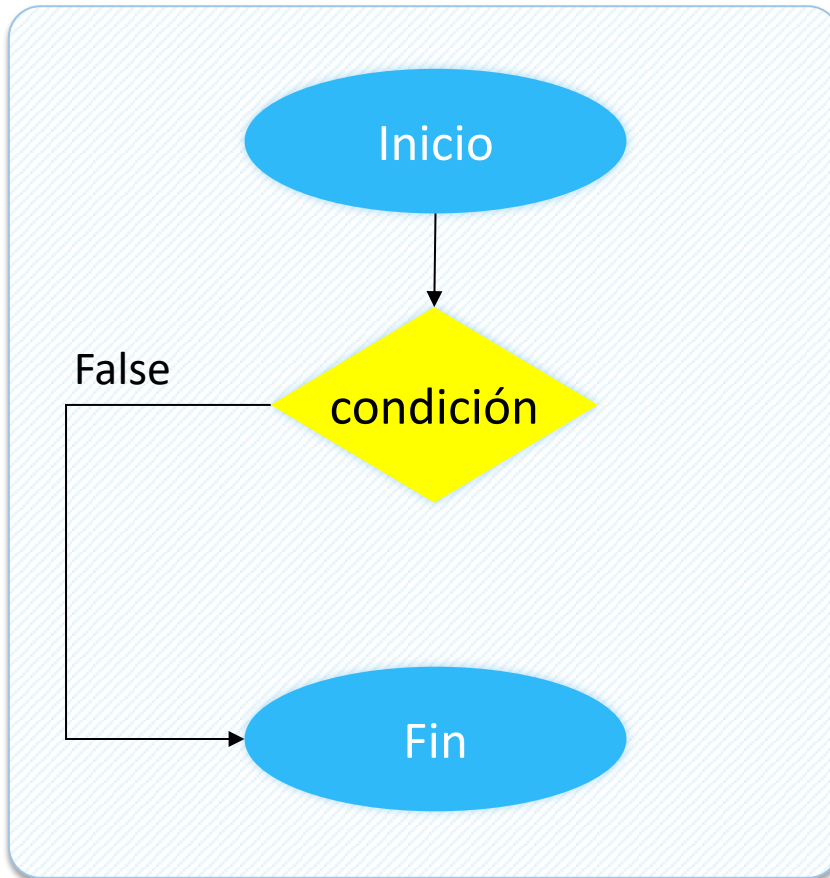
item_FidValue = OrderedDict.fromkeys(item_FidValue.keys())
#Read item in dictionary
for key, value in item_FidValue.items():
    typeOfFID = mapFidType[str key]
    if (typeOfFID == "DATE"):
        d = datetime.datetime.strptime(str value, "%Y-%m-%d")
        dataCal = datetime.datetime.strptime(str value, "%Y-%m-%d")
        FidAndValue = FidAndValue + (key, value)
    else:
        FidAndValue = FidAndValue + (key, value)

```

64

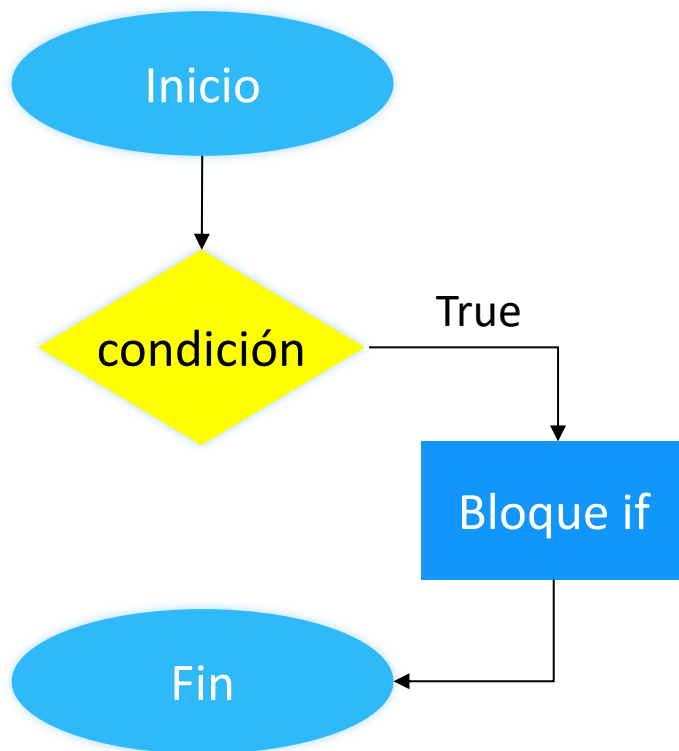
if (<condición lógica>):

Si la condición es verdadera (True o un entero positivo), el control del programa se dirige al bloque de instrucciones del **if**. Luego continúa su ejecución fuera del bloque



if (<condición lógica>):

Si la condición es verdadera, el control del programa se dirige al bloque de instrucciones del **if**. Luego continúa su ejecución fuera del bloque



```
area = 10
```

```
w = 8
```

```
if area > 10:
```

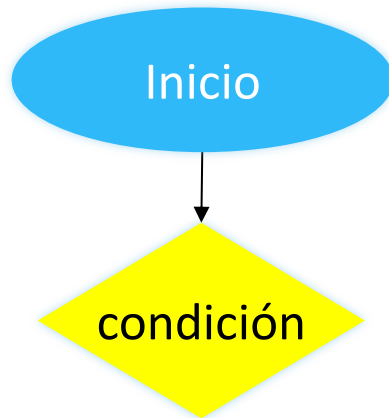
bloque IF

```
area = 9
```

bloque de
instrucciones

Importante

Las instrucciones dentro del bloque deben estar **indentadas** con al menos un espacio. Si no lo hace, el intérprete no ejecutará el programa

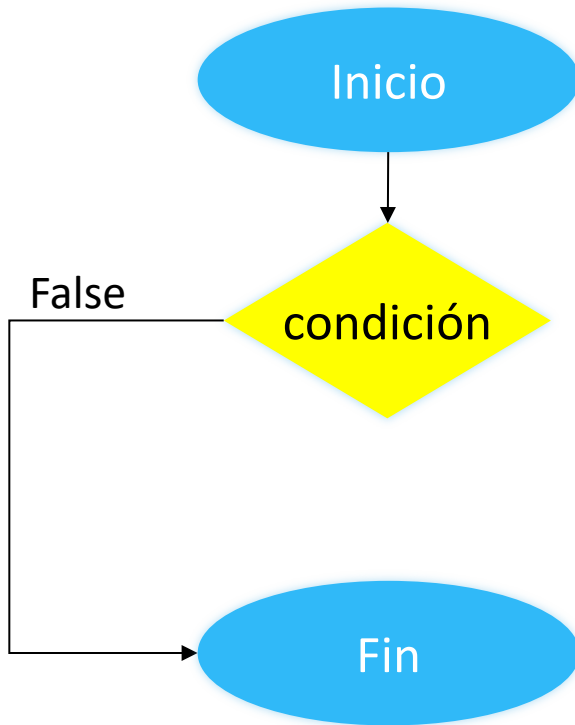


```
area = 10
w = 8

if area > 10:
    print('El area es mayor a 10')
    w = 11

area = 9
```

¿Qué valor tiene la variable **w** al término del programa?



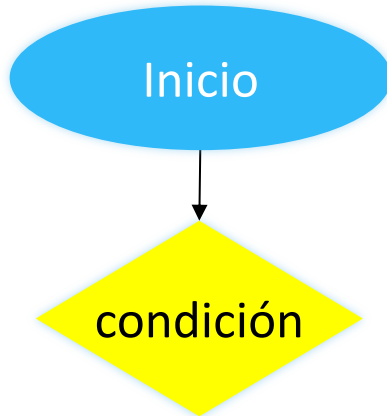
```
area = 10
w = 8

if area > 10:
    print('El area es mayor a 10')
    w = 11

area = 9
```

¿Qué valor tiene la variable w al término del programa?

la variable w tiene el valor 8 ya que NO ingresa al bloque IF, y NO se modifica el valor asignado al comienzo del programa.



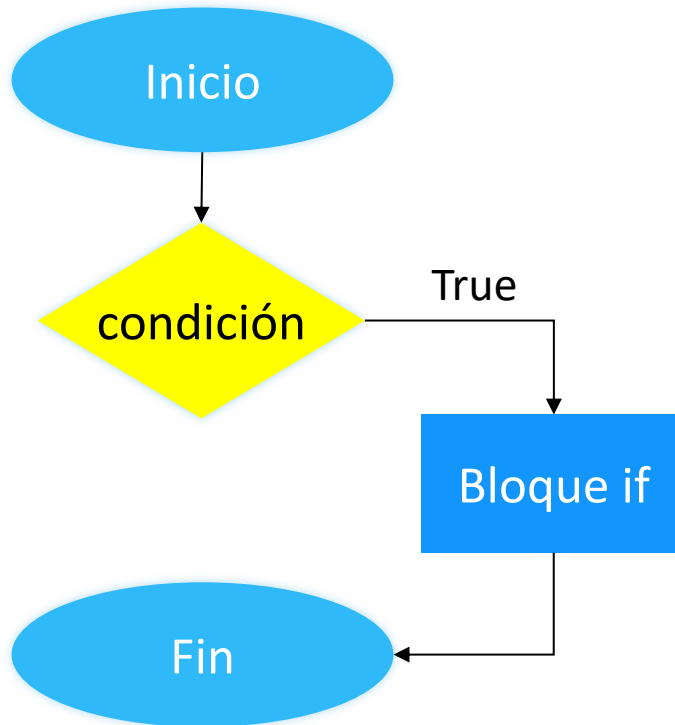
```
area = 10
w = 8

if area >= 10:
    print('El area es mayor a 10')
    w = 11

area = 9
```

condición

¿Qué valor tiene la variable w al término del programa?



```
area = 10  
w = 8
```

```
if area >= 10:  
    print('El area es mayor a 10')  
    w = 11
```

```
area = 9
```

¿Qué valor tiene la variable w al término del programa?

la variable w el valor 11 ya que ingresa al bloque IF, y en ella se modifica el valor almacenado en la variable w.

▼ if(condición):

La instrucción **if()**: nos permite evaluar una **condición lógica** que tiene dos resultados posibles: **verdadero** o **falso**.

The screenshot shows a Python REPL interface. On the left, a file named 'main.py' contains the following code:

```
1 area = 10
2 w= 8
3
4 if ( area > 10 ):
5     print("El area es mayor a 10")
6     w = 11
7
8 area = 9
9 print(str(w))
```

On the right, the terminal output shows the Python version and GCC version, followed by a prompt. A blue callout box points to the terminal with the text: "¿Qué resultado se muestra por la terminal?"

Tabla de comparaciones lógicas

comparación	==	distinto de	!=
mayor	>	menor	<
mayor igual	>=	menor igual	<=

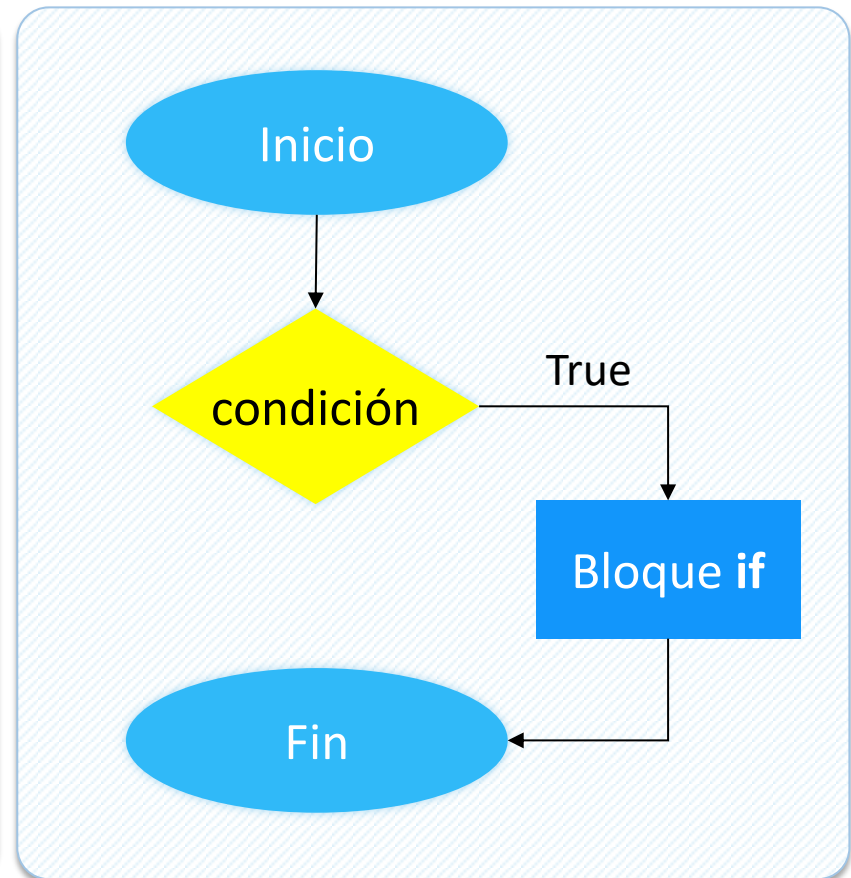
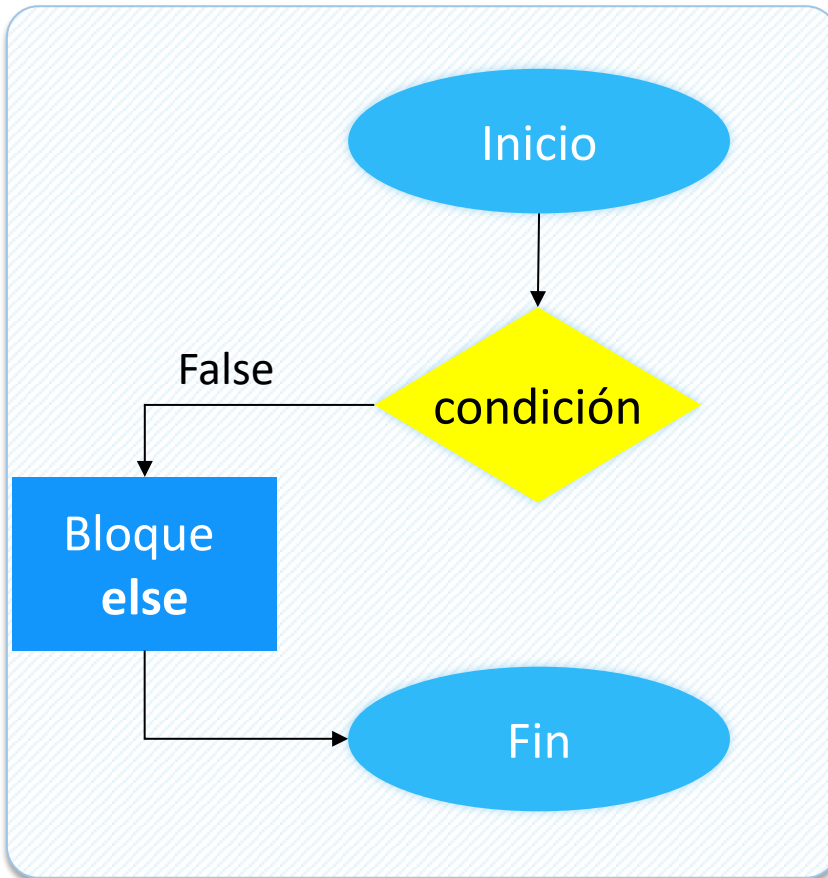
Resultados posibles

1	verdadero (TRUE)
0	falso (FALSE)

if (<condición lógica>):

else:

Si la condición es **verdadera**, el control del programa se dirige al bloque de instrucciones del **if**. Si es falsa realiza el bloque **else**.

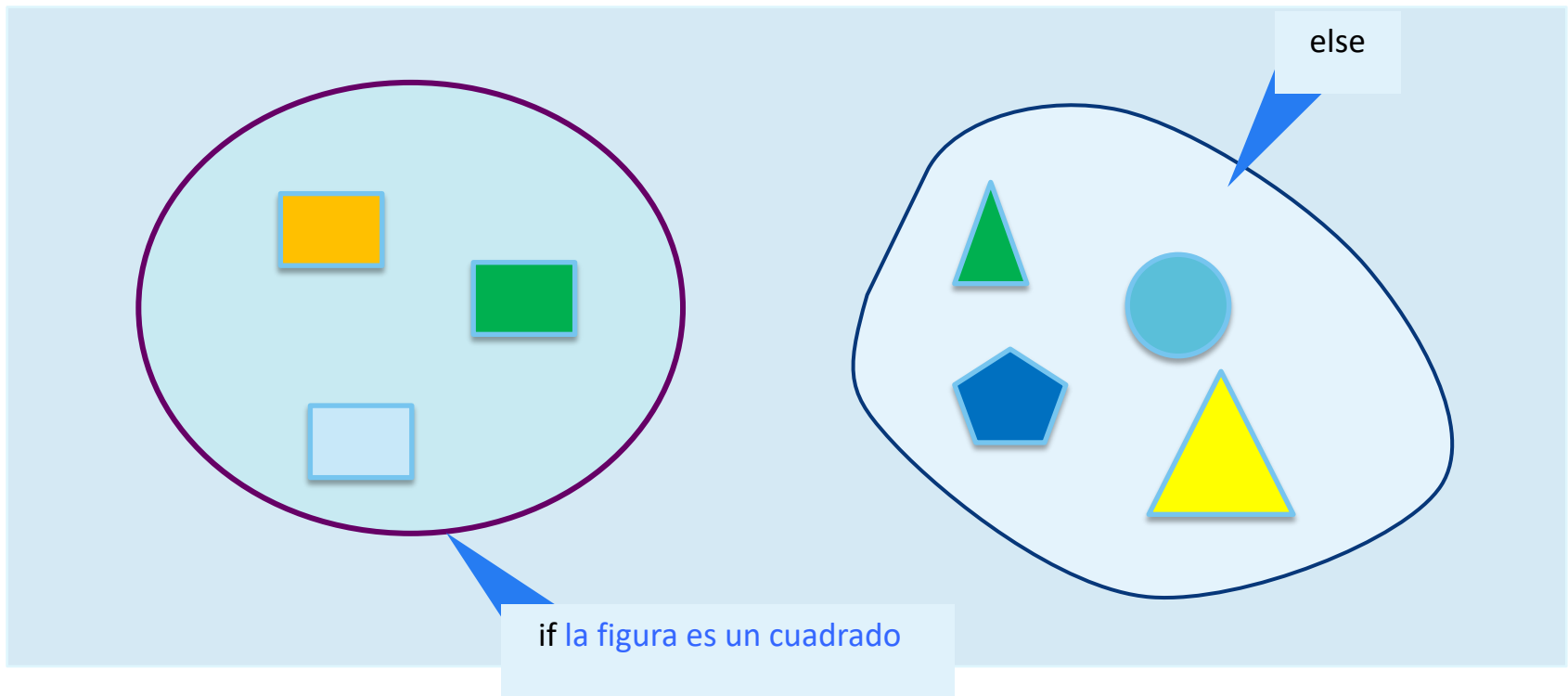


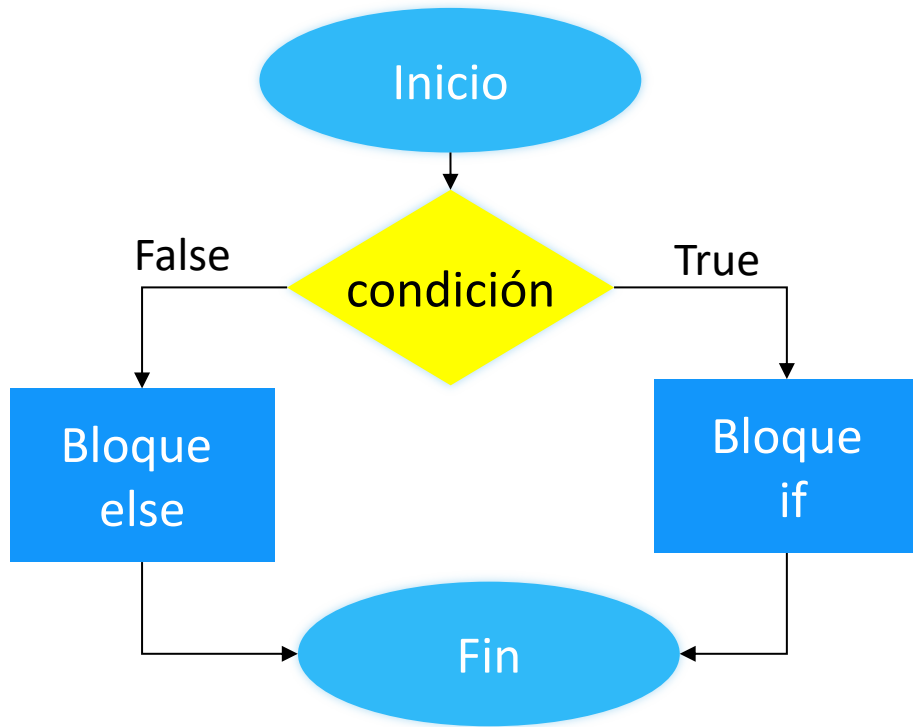
if (<condición lógica>):

else:

Si la condición es **verdadera**, el control del programa se dirige al bloque de instrucciones del **if**. Si es falsa realiza el bloque **else**.

Conjunto Universo = Figuras Geométricas





```
area = 10  
w = 8
```

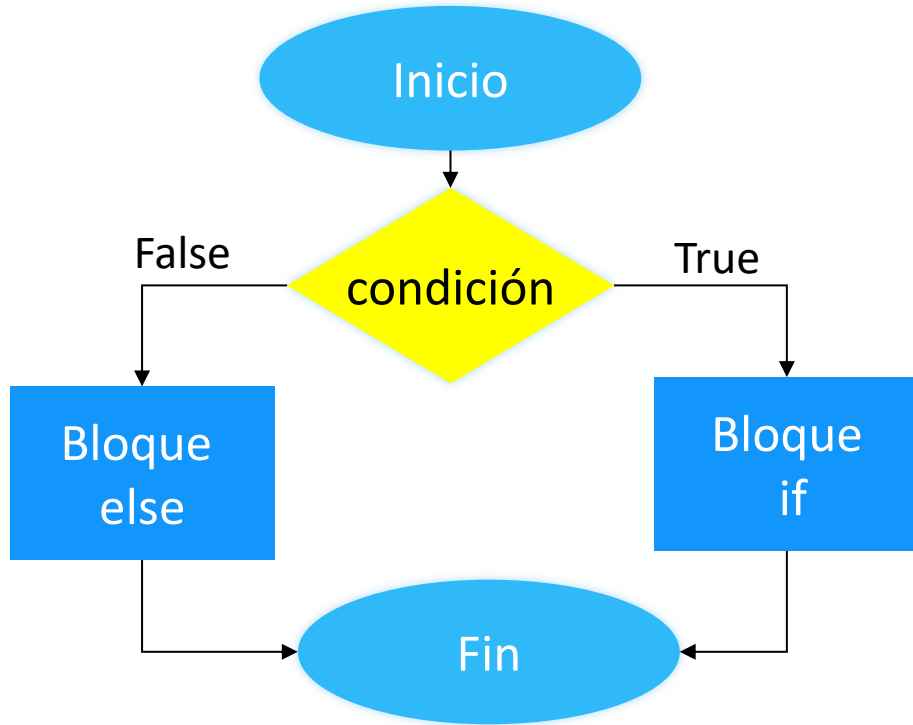
```
if area > 10:
```

bloque IF

```
else:
```

bloque ELSE

```
print(w)
```

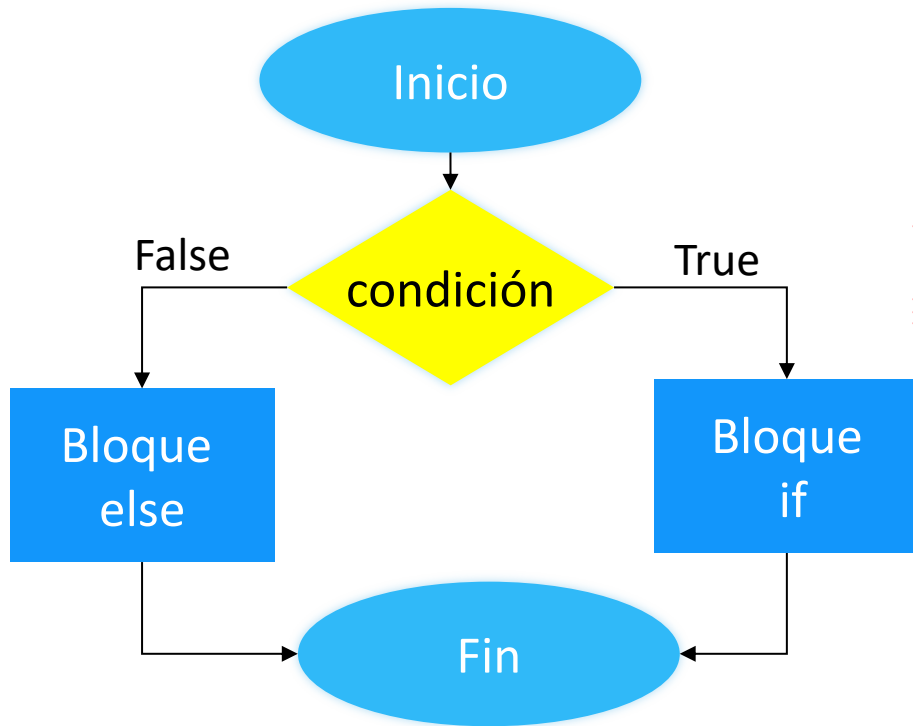
```
area = 10  
w = 8
```

```
if area > 10:  
    print('El area es mayor a 10')  
    w = 11
```

```
else:  
    area = 9  
    w = 7
```

```
print(w)
```

¿Qué valor tiene la variable *w* al término del programa?



```
area = 10
w = 8

if area > 10:
    print('El area es mayor a 10')
    w = 11
else:
    area = 9
    w = 7

print(w)
```

A red dashed arrow points from the text 'al bloque else' to the 'else:' line in the code.

¿Qué valor tiene la variable **w** al término del programa?

la variable **w** tiene el valor 7 ya que NO ingresa al bloque if, pero SI ingresa al bloque else, modificando su valor



EMPUJE LA PUERTA
SI SE ABRE,
ESTA ABIERTO
CASO CONTRARIO, NO

-
- A blue background featuring a white grid. In the upper center is the Python logo, composed of two interlocking snakes, one blue and one yellow. Below the logo is a white candlestick chart with vertical lines representing price movement. At the bottom, there is a white line graph and a bar chart with blue bars of varying heights.

```
item.FidValue = OrderedDict.fromkeys(item.FidValue.keys(), 0)
# Read item in dictionary
for key, value in item.FidValue.items():
    typeOfFid = mapFidType[str key]
    if (typeOfFid == "DATE"):
        d = datetime.datetime.strptime(str value, "%Y-%m-%d")
        dataCal = datetime.date.today() - d
        FidAndValue = FidAndValue + str key + str value + "\n"
    else:
        FidAndValue = FidAndValue + str key + str value + "\n"
```

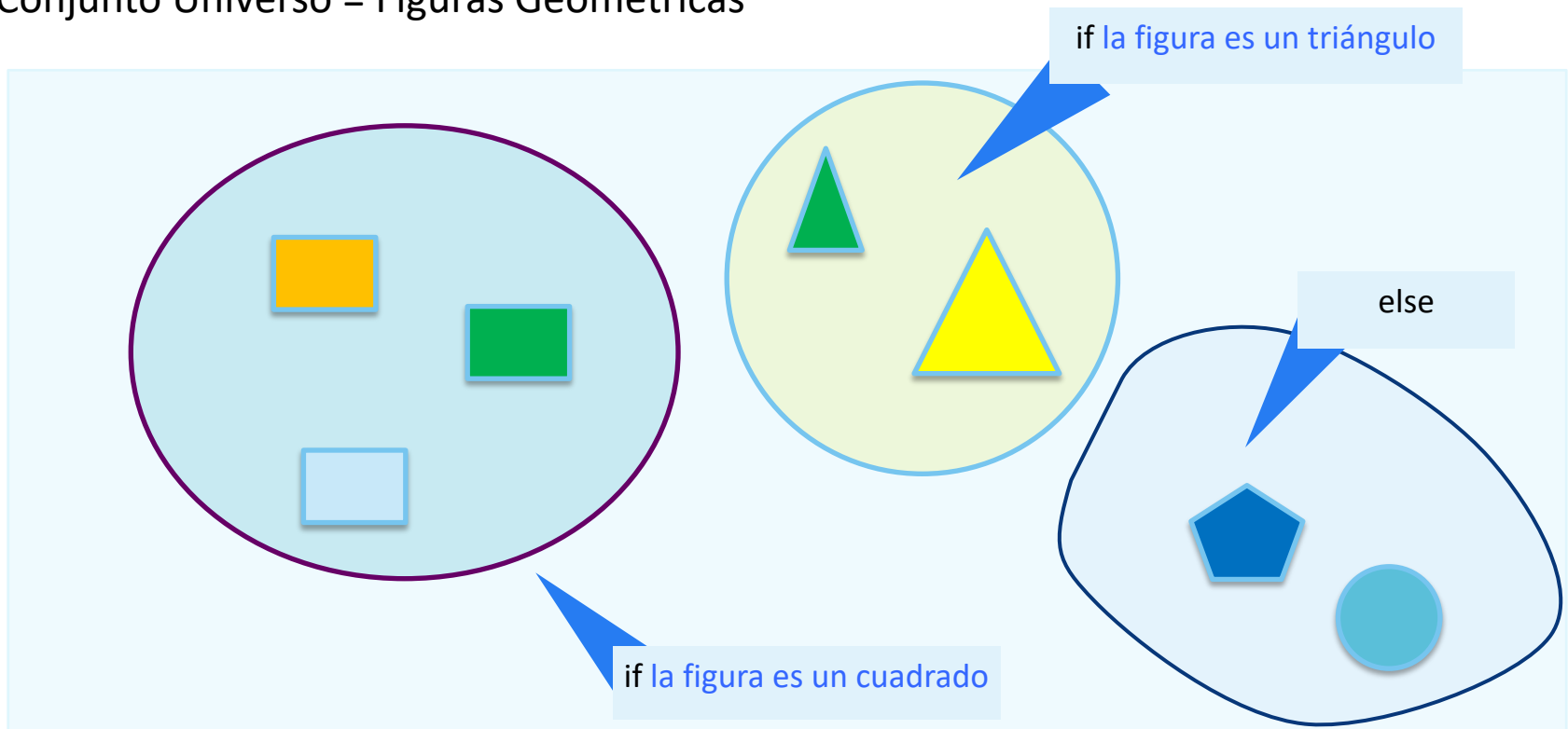
```
try:
    start = date(int(self.start_year.get()),
                  self.months.index(self.start_month.get()),
                  int(self.start_day.get()))

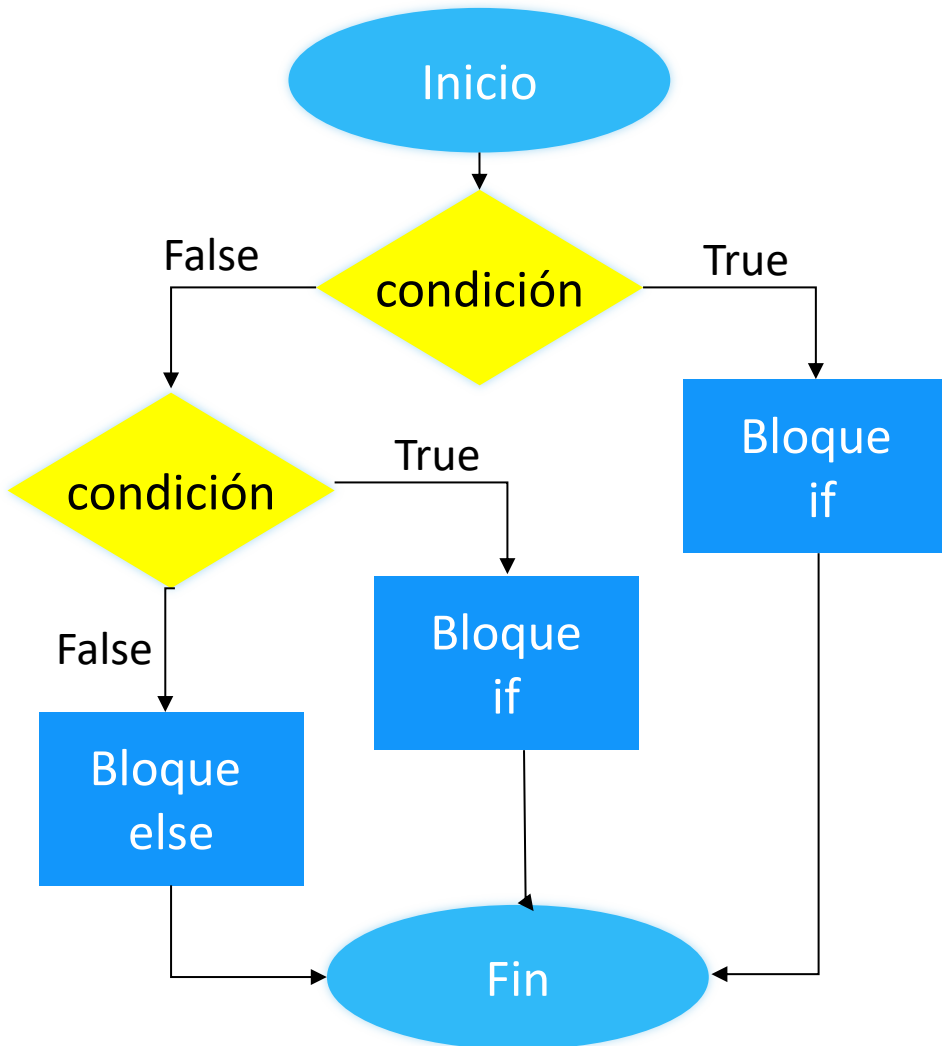
    end = date(int(self.end_year.get()),
                self.months.index(self.end_month.get()),
                int(self.end_day.get()))
```



Hay veces que tenemos que separar nuestro conjunto universo en más de dos grupos: cuadrados, triángulos y el resto
¿Cómo se escribirán este tipo de condiciones en Python?

Conjunto Universo = Figuras Geométricas





```
area = 5  
w = 8
```

```
if area > 10:
```

bloque if

```
else:
```

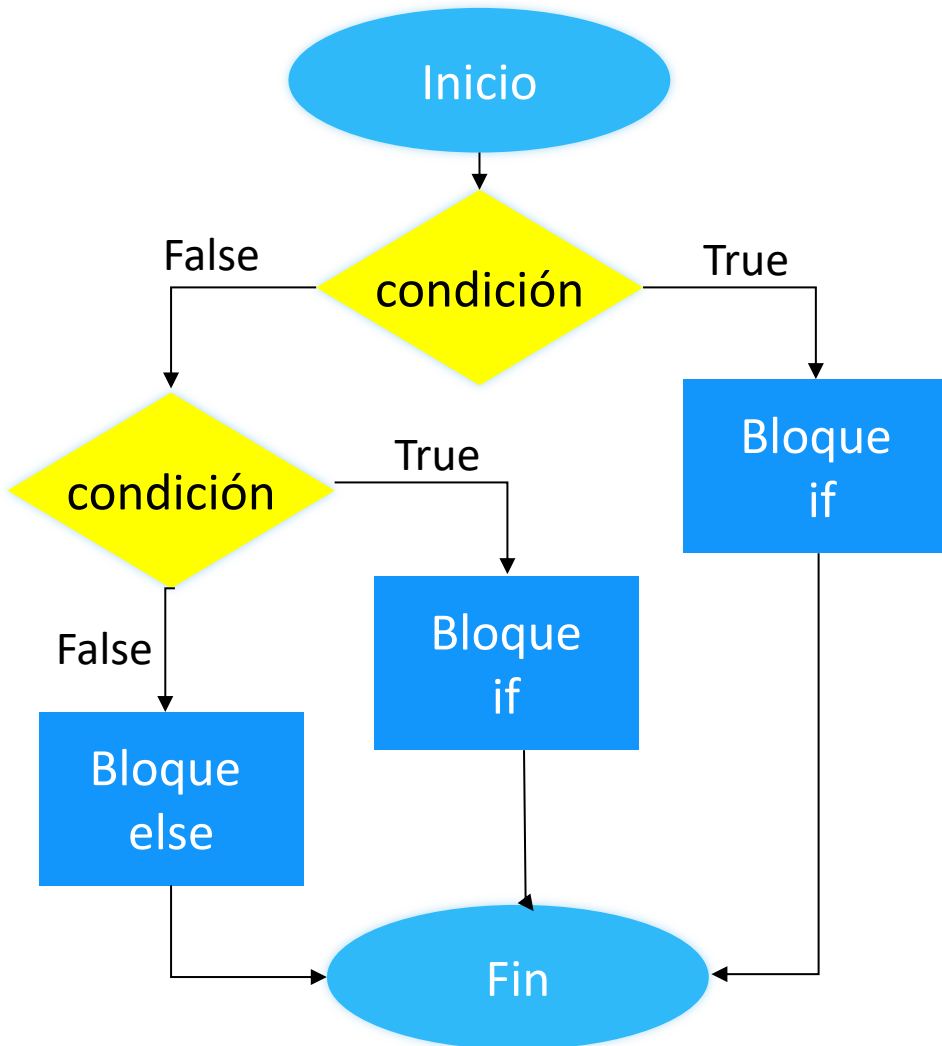
```
if area > 5:
```

bloque else if

```
else:
```

bloque else

```
print(w)
```



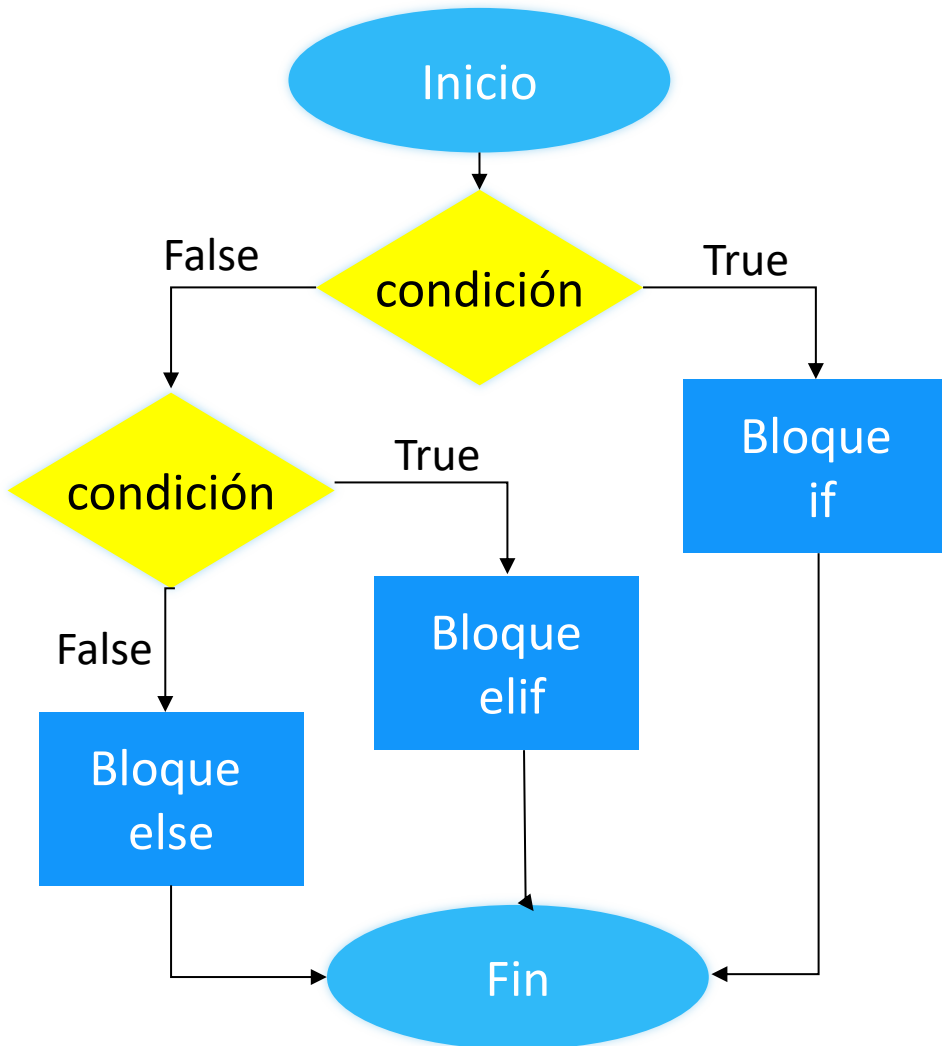
```
area = 5
w = 8

if area > 10:
    print('El area es mayor a 10')
    w = 11
else:
    if area > 5:
        print('El area es mayor a 5')
        w = 6
    else:
        print('El area es menor a 6')
        w = 1
print(w)
```

¿se fijaron que hay que indentar más?

¿Qué valor tiene la variable **w** al término del programa?

En Python podemos compactar las condiciones anidadas haciendo uso de **elif SIEMPRE** y **CUANDO** sean excluyentes



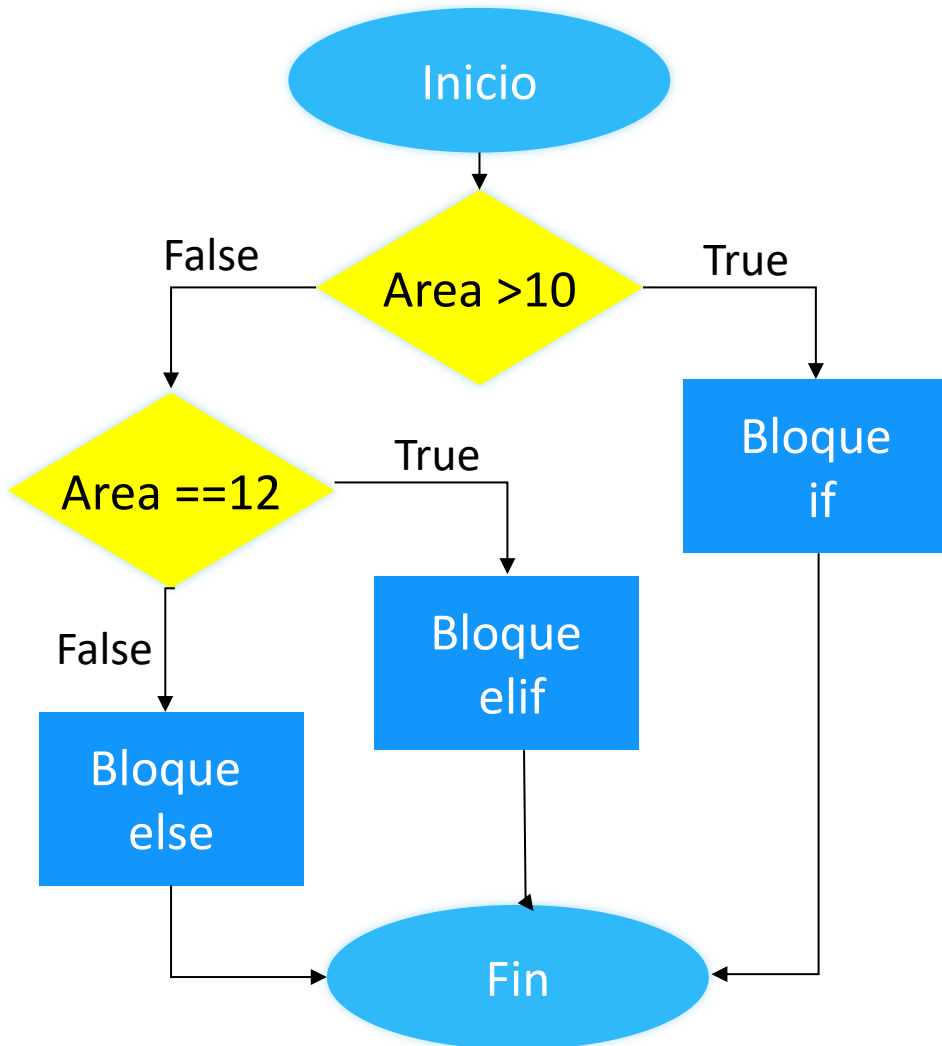
```
area = 5
w = 8

if area > 10:
    print('El área es mayor a 10')
    w = 11

elif area > 5:
    print('El área es mayor a 5')
    w = 6

else:
    print('El área es menor a 6')
    w = 1

print(w)
```

```
area = 5
w = 8

if area > 10:
    print('El área es mayor a 10')
    w = 11

elif area == 12:
    print('El área es 12')
    w = 6

else:
    print('área es menor o igual a 10')
    w = 1

print(w)
```

Esto no funciona..debe usarse:



```
area = 5
w = 8

if area > 10:
    print("El area es mayor o a 10")
    w = 11

elif area == 12:
    print("El area es 12")
    w = 6

else:
    print("area es menor o igual a 10")
    w = 1

print(w)
```

```
area = 5
w = 8

if area > 10:
    if area == 12:
        print("El area es 12")
        w = 6
    else:
        print("El area es mayor a 10")
        w = 11

else:
    print("area es menor o igual a 10")
    w = 1

print(w)
```



```
bool result;  
  
if (value_1 == true && value_2 == true)  
    result = true;  
  
else if (value_1 == true && value_2 == false)  
    result = false;  
  
else if (value_1 == false && value_2 == true)  
    result = false;  
  
else if (value_1 == false && value_2 == false)  
    result = true;  
  
if (result == true)  
    return true;  
  
else if (result == false)  
    return false;
```



```
return value_1 == value_2;
```

@khaby.lame

-
- A blue background featuring a white grid. In the upper center is the Python logo, composed of two interlocking snakes, one blue and one yellow. Below the logo is a white candlestick chart with vertical lines representing price movement. At the bottom, there is a white line graph and a bar chart with blue bars of varying heights.

```
item.FidValue = OrderedDict.fromkeys(item.FidValue.keys(), 0)
# Read item in dictionary
for key, value in item.FidValue.items():
    typeOfFid = mapFidType[str key]
    if (typeOfFid == "DATE"):
        d = datetime.datetime.strptime(str value, "%Y-%m-%d")
        dataCal = datetime.date.today() - d
        FidAndValue = FidAndValue + str key + str value + "\n"
    else:
        FidAndValue = FidAndValue + str key + str value + "\n"
```

```
try:
    start = date(int(self.start_year.get()),
                  self.months.index(self.start_month.get()),
                  int(self.start_day.get()))

    end = date(int(self.end_year.get()),
                self.months.index(self.end_month.get()),
                int(self.end_day.get()))
```



Tiempo : 15 minutos

Realice un programa que le pregunte al usuario si le gustan los dulces y muestre el mensaje “A mi también” si escribe algunas de las siguientes respuestas: { Sí, Si, si, sí }



▼ Preguntas

- ¿Cómo solicito los datos al usuario?*
- ¿Cómo creo una condición compuesta?*
- ¿Cómo se muestra el resultado?*



Tiempo : 15 minutos

Realice un programa que le pregunte al usuario si le gustan los dulces y muestre el mensaje “A mi también” si escribe algunas de las siguientes respuestas: { Sí, Si, si, sí }

Solución propuesta #1

```
print('Le gustan los dulces')
rsp= input()

if rsp == 'Si' or rsp == 'Sí' or rsp == 'si' or rsp == 'sí':
    print('A mi tambien')
```

Solución propuesta #2

```
if input('Te gustan los dulces') in ('Sí', 'Si', 'si', 'sí'):
    print("A mi tambien")
```



Tiempo : 5 minutos

Realice un programa que pregunte un número al usuario y luego imprima en pantalla si el número es par o impar



▼ Preguntas

¿Cómo solicito los datos al usuario?

¿Cómo sé que un número es par?

¿Cómo se muestra el resultado?



Tiempo : 5 minutos

Realice un programa que pregunte un número al usuario y luego imprima en pantalla si el número es par o impar

Solución propuesta

```
print('Ingresa un número')
num = int(input())

if num%2==0:
    print('El numero es par')
else:
    print('El numero es impar')
```




Tiempo : 15 minutos

Twitter tiene un límite de 28 caracteres. Realice un programa que permita al usuario ingresar un texto e imprimir por pantalla “Escribe algo” si el usuario no escribió un texto, “OK” si el texto tiene 28 caracteres o menos, o “Texto muy largo” en caso contrario



▼ Hint

Usa la función `len()` para obtener el largo de un texto

```
saludo = 'hola'  
largo = len(saludo)  
print('El largo de', saludo, 'es', largo)
```

Imprimirá en pantalla ‘El largo de hola es 4’



Tiempo : 15 minutos

Twitter tiene un límite de 28 caracteres. Realice un programa que permita al usuario ingresar un texto e imprimir por pantalla “Escribe algo” si el usuario no escribió un texto, “OK” si el texto tiene 28 caracteres o menos, o “Texto muy largo” en caso contrario

Solución propuesta

```
print('Escribe un mensaje')
texto = input()
largo = len(texto)

if largo == 0:
    print('Escribe algo')
elif largo > 0 and largo < 28:
    print('OK')
else:
    print('Texto muy largo')
```